

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 9. April 2026

Ertan Ertas

Kaiserslautern, 9. April 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	2
1.3	Methodik und Abgrenzung	3
2	Theoretische Grundlagen der Typisierung	5
2.1	Semantik und Funktion von Datentypen	5
2.2	Statische Typisierung mittels TypeScript	6
2.2.1	Herausforderungen einer naiven Code-Migration	8
2.2.2	Explizite Typannotationen	11
2.2.3	Strukturelles Typing und Interface-Verträge	16
2.3	Fazit zur architektonischen Neuausrichtung	23
3	RecursiveSet: API und fachliche Modellierung	25
3.1	Mathematische Mengen und Wohlunterscheidbarkeit	25
3.2	Kernbausteine der Bibliothek	26
3.2.1	Die Datenstruktur RecursiveSet	26
3.2.2	Strukturierte Elemente: Die Tuple-Klasse	27
3.2.3	Assoziative Daten: Die RecursiveMap	28
3.3	Verarbeitung und Lebenszyklus strukturierter Daten	29
3.3.1	Der Freeze-Contract: Immutabilität sichern	29
3.3.2	Klassische Mengenoperationen	30
3.3.3	Teil- und Obermengenbeziehungen	31
3.3.4	Auswahl eines beliebigen Elements	32
3.3.5	Kartesisches Produkt und Potenzmenge	32
3.3.6	Funktionale Transformationen statt Komprehension	33
4	Interne Architektur von RecursiveSet	37
4.1	Architektonische Grundentscheidungen	37
4.2	Die Hash-Engine (Zentrale Komponente)	38
4.2.1	Ganzzahlen und der ArrayBuffer-Trick für IEEE-754	38
4.2.2	Hashing von Strings und strukturellen Objekten	41
4.3	Speicherlayout (Structure of Arrays)	43
4.4	Konzepte der Gleichheit und Type Narrowing	45
4.5	Kollisionsauflösung und Index-Verwaltung	47
4.5.1	Open Addressing mit Linear Probing	47
4.5.2	Löschvorgang: Tombstones und Backshift Deletion	49
4.6	Dynamische Kapazitätsverwaltung (Resizing und Rehashing)	52

4.7	Technische Umsetzung des Freeze-Contracts	54
5	Vom Python-Werkzeug PLY zu Lezer	56
5.1	Grundlagen von Lezer	58
5.2	Grammatikdefinition in Lezer	60
5.3	Generierung des Parsers	61
5.4	Integration in die TypeScript-Implementierung	62
6	Case Study: Umsetzung eines vollständigen Interpreters in TypeScript	65
6.1	Ziel und Aufbau des Notebooks	65
6.2	Sprachdefinition und Parsing im Vergleich: Python (PLY) vs. TypeScript (Lezer)	66
6.3	Transformation vom CST zum AST	73
6.4	Darstellung rekursiver Listenstrukturen	74
6.5	Typisierung und interne AST-Repräsentation	75
6.6	Interpreterarchitektur und Ausführung	76
6.7	Auswertung arithmetischer Ausdrücke	79
6.8	Auswertung boolescher Bedingungen	82
6.9	Umgebungsmodell und Programmausführung	84
6.10	Ergebnisse der Case Study	85
7	Fazit	87
8	Literaturverzeichnis	89
A	Anhang	90
A.1	Verzeichnis der portierten Lehrnotebooks	90

Abbildungsverzeichnis

Tabellenverzeichnis

1	Übersicht der übersetzten Notebooks im Repository	90
---	---	----

List of Listings

1	Mengen in Python: Das erwartete Verhalten	2
2	Mengen in TypeScript: Das fehlerhafte Verhalten	2
3	Unterschiedliche Operationen erfordern unterschiedliche Typen . .	5
4	Dynamische Typisierung in Python	6
5	Dynamische Typisierung in JavaScript	6
6	Typfehler wird erst zur Laufzeit sichtbar	6
7	Eine untypisierte Funktion in JavaScript	7
8	Dieselbe Funktion in TypeScript mit expliziten Typen	7
9	Naive Übersetzung einer Funktion nach TypeScript	8
10	Typprüfung zur Laufzeit in Python	8
11	Typprüfung zur Laufzeit in JavaScript und TypeScript	8
12	Der Typ <code>any</code> hebt die Typprüfung aus	9
13	Explizite Umwandlung in Python	9
14	Implizite und explizite Umwandlung in JavaScript	10
15	Eine Type Assertion ist keine echte Umwandlung	10
16	Unpräzise vs. präzise formulierte Funktion in TypeScript	11
17	Atomare und zusammengesetzte TypeScript-Typen	12
18	Union Type aus Literal Types in TypeScript	13
19	Literal Types in Python	13
20	Ein möglicher fehlender Rückgabewert in TypeScript	13
21	Möglicher fehlender Rückgabewert in Python	14
22	Fehlerhafte Nutzung ohne vorherige Prüfung	14
23	Explizite Behandlung eines möglichen <code>undefined</code> -Werts	15
24	Der Typ <code>unknown</code> erzwingt eine vorherige Prüfung	15
25	Einengung von <code>unknown</code> durch Typprüfung im Kontrollfluss	16
26	Type Hints in Python	16
27	Python <code>Any</code> vs. TypeScript <code>unknown</code>	17
28	Ein Interface als Vertrag über benötigte Methoden	17
29	Eine Klasse erfüllt ein Interface über ihre reine Struktur	18
30	Ein Protocol in Python als statischer Vertrag	18
31	Feste Datenstruktur mit <code>TypedDict</code> in Python	19
32	Objektstrukturen und Aliase: <code>interface</code> und <code>type</code> in TypeScript . .	20
33	Eine generische Funktion in TypeScript	20
34	Eine generische Funktion in Python	21
35	Generics mit strukturellen Schranken im Vergleich	21
36	Type Narrowing durch Kontrollfluss und <code>typeof</code>	22
37	Klassenprüfung mit <code>instanceof</code>	22

38	Vollständige Fallunterscheidung mit <code>never</code>	23
39	Mengenverhalten in Python	25
40	Mengenverhalten in TypeScript	26
41	Hinzufügen von Elementen in Python und mit <code>RecursiveSet</code>	27
42	Mitgliedschaftstest in Python und mit <code>RecursiveSet</code>	27
43	Entfernen von Elementen in Python und mit <code>RecursiveSet</code>	28
44	Roboter-Koordinaten: Tupel in Python vs. TypeScript	28
45	Dictionaries in Python vs. <code>RecursiveMap</code> in TypeScript	29
46	Der Freeze-Contract bei geschachtelten Mengen	30
47	Arbeiten mit einer veränderlichen Kopie	30
48	Vereinigung zweier Mengen in Python und mit <code>RecursiveSet</code>	31
49	Schnittmenge zweier Mengen in Python und mit <code>RecursiveSet</code>	31
50	Differenz zweier Mengen in Python und mit <code>RecursiveSet</code>	31
51	Symmetrische Differenz in Python und mit <code>RecursiveSet</code>	32
52	Teil- und Obermengenbeziehungen im Sprachvergleich	32
53	Auswahl eines Elements in Python und mit <code>RecursiveSet</code>	33
54	Kartesisches Produkt in Python und mit <code>RecursiveSet</code>	33
55	Potenzmenge mit <code>RecursiveSet</code>	34
56	Python: Mengenkompensation mit Selektion und Transformation	34
57	Selektion mit <code>filter</code>	34
58	Abbildung mit <code>map</code>	35
59	Logische Quantoren mit <code>every</code> und <code>some</code>	35
60	Kombination aus Filter und Abbildung mit <code>filterMap</code>	35
61	Python: Verschachtelte Mengenkompensation	36
62	Verschachtelte Konstruktion mit <code>flatMap</code>	36
63	Aggregation mit <code>reduce</code>	36
64	Hashing von numerischen Werten	39
65	Hashing von Strings und strukturellen Objekten	41
66	Der Vertrag für komplexe Mengenelemente	42
67	Speicherarchitektur im Structure of Arrays (SoA) Format	43
68	Die <code>equals</code> -Methode von <code>RecursiveSet</code>	46
69	Linear Probing am Beispiel der <code>has</code> -Methode	48
70	Reparatur der Sondierungskette (Backshifting)	50
71	Der Rehashing-Prozess bei einer Kapazitätsänderung	53
72	Einfrieren durch Hash-Abruf	54
73	Ply-Grammatik in Python	57
74	Lezer-Grammatik in TypeScript	58
75	Umsetzung Lezer-Grammatik in TypeScript	61
76	Erstellung des Parsers in TypeScript	62

77	Vollständige Integrationslogik in TypeScript	63
78	Die typsichere Definition des AST	64
79	Auszug der PLY-Scannerdefinition (Python)	66
80	Direkte AST-Konstruktion in PLY (Auszug)	67
81	Beginn der Lezer-Grammatik (Startsymbol und Scanner-Token) . .	69
82	Fortsetzung der Lezer-Grammatik (Teil 1: Anweisungen)	71
83	Ende der Lezer-Grammatik (Teil 2: Ausdrücke)	72
84	CST-zu-AST-Transformation in TypeScript	74
85	Verarbeitung rekursiver Listen in TypeScript	75
86	AST-Typmodellierung in Python	76
87	AST-Typdefinition in TypeScript	76
88	Ausführung von Anweisungen in Python	77
89	Ausführung von Anweisungen in TypeScript	78
90	Arithmetische Auswertung in Python	79
91	Arithmetische Auswertung in TypeScript (Teil 1: Basisoperatoren) .	80
92	Arithmetische Auswertung in TypeScript (Teil 2: Funktionsaufrufe) .	81
93	Boolesche Auswertung in Python	82
94	Boolesche Auswertung in TypeScript	83
95	Programmausführung in Python	84
96	Programmausführung und I/O-Simulation in TypeScript	84

1 Einleitung

Die Theoretische Informatik bildet ein wichtiges Fundament im Informatikstudium. Themen wie Automaten oder formale Sprachen können anfangs jedoch sehr abstrakt wirken. Um diese theoretischen Konzepte greifbarer zu machen, kommen in der Vorlesung „Theoretische Informatik III“ interaktive Notebooks zum Einsatz. Dort wird erklärender Text direkt mit ausführbarem Programmcode gemischt. Bislang basieren diese Notebooks auf der Programmiersprache Python. Studierende können so theoretische Algorithmen direkt ausprobieren, verändern und live nachvollziehen.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in die Programmiersprache TypeScript. Der Fokus liegt dabei auf einer Portierung, welche nicht nur die Syntax übersetzt, sondern die Konzepte der formalen Sprachen durch feste, verbindliche Datenstrukturen explizit modelliert.

1.1 Problemstellung

Ausgangspunkt dieser Arbeit sind interaktive Python-Notebooks, die Konzepte der theoretischen Informatik demonstrieren. Werden diese Lehrmaterialien in das TypeScript-Ökosystem übertragen, reicht es jedoch nicht aus, lediglich die Syntax der Sprache zu übersetzen. Es entstehen fundamentale technische Hürden, da beide Sprachen Daten auf unterschiedliche Weise verarbeiten. Folgende zwei Problemfelder stehen dabei im Fokus:

Zu späte Fehlererkennung bei verschachtelten Daten

Python ist eine dynamische Sprache. Das bedeutet, dass Variablen keine festen Typen haben und Fehler oft erst dann auffallen, wenn das Programm ausgeführt wird und abstürzt. Zwar gibt es Werkzeuge, um Typen in Python zu prüfen, in der interaktiven Umgebung eines Jupyter-Notebooks übersehen diese jedoch häufig Fehler.

In der Vorlesung arbeiten wir ständig mit tief verschachtelten Objekten – beispielsweise Datenstrukturen, bei denen eine Liste wieder viele weitere Objekte und Listen enthält. Fehlt einem dieser verschachtelten Elemente versehentlich eine wichtige Eigenschaft, merkt Python das erst, wenn der Code im Hintergrund genau dieses fehlerhafte Element aufruft. Wir benötigen stattdessen eine Sprache, die uns schon während des Tippens garantiert, dass unsere Datenstrukturen über alle Ebenen hinweg fehlerfrei und vollständig aufgebaut sind.

Das unerwartete Verhalten von Mengen (Sets)

Das gravierendste Hindernis bei der Übersetzung zeigt sich jedoch bei der Arbeit mit Mengen (Sets). In der theoretischen Informatik ist es unumgänglich, berechnete Zustände in Mengen zu speichern, um Duplikate zu vermeiden oder später zu prüfen, ob ein Zustand bereits erreicht wurde.

In Python funktioniert das exakt so, wie man es intuitiv erwartet. Fügen wir ein Tupel in eine Menge ein und fragen danach ab, ob ein Tupel mit denselben Zahlen enthalten ist, liefert Python das erwartete Ergebnis:

```
1  menge = set()
2  menge.add((1, 2))
3
4  # Die Abfrage liefert das erwartete Ergebnis
5  print((1, 2) in menge) # Ausgabe: True
```

Listing 1: Mengen in Python: Das erwartete Verhalten

Ein naiver Übersetzungsversuch nach TypeScript führt hier jedoch zu einem fundamentalen Bruch in der Logik. Nutzen wir die Standard-Datenstruktur Set in TypeScript auf die exakt gleiche Weise, erhalten wir plötzlich ein falsches Ergebnis:

```
1  const menge = new Set();
2  menge.add([1, 2]);
3
4  // TypeScript findet das Array überraschenderweise nicht
5  console.log(menge.has([1, 2])); // Ausgabe: false
```

Listing 2: Mengen in TypeScript: Das fehlerhafte Verhalten

Obwohl das Array `[1, 2]` im Code zuvor offensichtlich in die Menge eingefügt wurde, behauptet TypeScript, es sei nicht vorhanden. Dieser zunächst unerklärliche Unterschied führt bei einer direkten Portierung dazu, dass Logik-Abfragen fehlschlagen, Endlosschleifen entstehen und Algorithmen falsche Ergebnisse liefern. Die Ursachenforschung und Behebung dieses Verhaltens ist die zentrale technische Herausforderung dieser Arbeit.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist es, die Lehrinhalte der Vorlesung „Theoretische Informatik III“ vollständig nach TypeScript zu übersetzen. Die neue Co-debasis soll die bisherigen Python-Notebooks ersetzen und dabei die klaren Strukturvorgaben der neuen Sprache nutzen, um Fehler frühzeitig zu vermeiden.

Daraus leiten sich folgende konkrete Teilziele ab:

1. **Strukturierte Neuimplementierung:** Die Algorithmen der Vorlesung sollen in TypeScript portiert werden. Die dynamische, oft fehleranfälligere Natur von Python soll dabei durch verbindliche Datenstrukturen und Typvorgaben ersetzt werden.
2. **Lösung des Mengen-Problems:** Um die Algorithmen fehlerfrei auszuführen, muss das unerwartete Verhalten von TypeScripts Set analysiert und behoben werden. Ziel ist die Entwicklung einer maßgeschneiderten Datenstruktur, die sich exakt wie ein Python-Set verhält und verschachtelte Objekte verlässlich erkennt.
3. **Einheitliche Ausführungsumgebung:** Da die Notebooks lokal ausgeführt werden, muss eine standardisierte Umgebung geschaffen werden, die unabhängig vom Betriebssystem identisch funktioniert. Ein wichtiges technisches Teilziel ist es dabei, die Hintergrundprozesse der Notebooks so anzupassen, dass TypeScripts strenge Prüfregele auch bei der interaktiven, schrittweisen Ausführung zuverlässig erzwungen werden. Der Einrichtungsaufwand für Studierende soll durch eine klare Schritt-für-Schritt-Anleitung minimal gehalten werden.

1.3 Methodik und Abgrenzung

Die methodische Durchführung dieser Arbeit folgt einem fokussierten Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die genaue Auswahl der zu übersetzenden Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Ein zentraler methodischer Aspekt betrifft die didaktische Aufbereitung: Die begleitenden Erklärtexpte der Notebooks wurden nicht isoliert übersetzt, sondern inhaltlich an die neuen Strukturen von TypeScript angepasst. Dabei wurde bewusst darauf verzichtet, die neuen Konzepte kontinuierlich mit der alten Python-Implementierung zu vergleichen. Die Notebooks sollen als eigenständiges Lehrmaterial funktionieren, das die theoretischen Konzepte nativ und in sich geschlossen vermittelt.

Auf technischer Ebene war die Minimierung von Systemabhängigkeiten ein wesentliches Entwurfsziel. Insbesondere für die grafische Visualisierung von Automaten und Graphen wurde darauf geachtet, keine komplexen lokalen Installationen

von externer Software vorauszusetzen. Stattdessen kommen integrierbare, plattformunabhängige Lösungen zum Einsatz, um die Ausführung auf verschiedenen Betriebssystemen zu garantieren und den Einstieg für die Studierenden so barrierefrei wie möglich zu gestalten.

Zur Gewährleistung der Reproduzierbarkeit und für die freie Zugänglichkeit der Ergebnisse wird der gesamte entwickelte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstrerath/formal-languages-typescript> im Verzeichnis TypeScript abrufbar.

2 Theoretische Grundlagen der Typisierung

Beim Übergang von den interaktiven Python-Notebooks der Vorlesung zu einer TypeScript-Implementierung ändert sich nicht nur die Syntax. Der fundamentalste Unterschied liegt im Umgang mit Datentypen. Die in Python erprobten Algorithmen wurden ursprünglich in einer hochdynamischen Umgebung entwickelt und mittels „mypy“ typisiert. TypeScript verwendet einen anderen Ansatz, die Typisierung ist der zentrale Aspekt der Sprache, daher ist ebenjene in diesem Projekt kein Randthema, sondern das tragende Fundament der gesamten Codebasis.

2.1 Semantik und Funktion von Datentypen

Im Arbeitsspeicher eines Computers bestehen alle Daten letztlich nur aus Bitfolgen, also aus Nullen und Einsen. Für den Prozessor ist eine solche Bitfolge zunächst bedeutungslos. Erst durch einen „Typ“ erhält sie eine fachliche Interpretation. Der Typ legt fest, ob ein Wert beispielsweise als Zahl, als Text, als Wahrheitswert oder als komplexe Datenstruktur behandelt wird.

Man kann einen Typ daher als eine Art Vertrag verstehen. Dieser Vertrag beschreibt die Form eines Wertes und definiert, welche Operationen auf ihm sinnvoll und erlaubt sind. Ein kurzes Python-Beispiel (siehe Listing 3) verdeutlicht dieses Prinzip: Eine Zahl mit einer anderen Zahl zu summieren ist logisch, ebenso die Verkettung zweier Strings. Die Addition von String und Zahl führt hingegen zu einem inhaltlichen Konflikt.

```
1 zahl = 42
2 text = "42"
3 print(zahl + 1)      # sinnvoll: 43
4 print(text + "!")    # sinnvoll: "42!"
5 print(text + 1)      # nicht sinnvoll: Text und Zahl passen hier nicht zusammen
```

Listing 3: Unterschiedliche Operationen erfordern unterschiedliche Typen

Ob eine Operation zulässig ist, hängt demnach maßgeblich vom Typ ab. Genau hier setzen Typsysteme an: Sie strukturieren die Bedeutung von Daten und dienen als essenzieller Sicherheitsmechanismus. Wenn ein Programm versucht, eine Operation auf einem ungeeigneten Wert auszuführen, liegt oft kein syntaktischer Fehler vor, sondern ein inhaltlicher Widerspruch. Ein Typsystem macht solche Widersprüche sichtbar, bevor sie zu schwer nachvollziehbaren Abstürzen im laufenden Programm führen.

Prinzipien der dynamischen Typisierung

Python und JavaScript sind dynamisch typisierte Sprachen. Das bedeutet, dass Variablen nicht dauerhaft an einen vorher festgelegten Typ gebunden sind. Der Typ ergibt sich stattdessen aus dem Wert, der zur Laufzeit gerade in der Variablen gespeichert ist. Wie Listing 4 zeigt, kann eine Variable in Python nahtlos nacheinander völlig unterschiedliche Datentypen aufnehmen.

```
1 value = 42
2 value = "Hallo"
3 value = [1, 2, 3]
4 print(value)
```

Listing 4: Dynamische Typisierung in Python

Dasselbe Grundprinzip findet sich unverändert in JavaScript wieder (siehe Listing 5), wenn man das Keyword `let` verwendet.

```
1 let value = 42;
2 value = "Hallo";
3 value = [1, 2, 3];
4 console.log(value);
```

Listing 5: Dynamische Typisierung in JavaScript

Diese Flexibilität erleichtert das schnelle Schreiben von Programmcode. Gleichzeitig verschiebt sie einen großen Teil der Verantwortung auf die Entwickelnden, da sich oft erst im Moment der Ausführung zeigt, ob eine Operation zum aktuellen Wert passt. Die möglichen Folgen dieser späten Auswertung demonstriert Listing 6.

```
1 value = [1, 2, 3]
2 print(value + 1)
```

Listing 6: Typfehler wird erst zur Laufzeit sichtbar

Obwohl der inhaltliche Fehler, die Addition von Liste und Ganzzahl, im Quelltext sofort ersichtlich ist, bemerkt Python ihn erst bei der Ausführung. In komplexen Algorithmen erschwert dies die Fehlererkennung erheblich.

2.2 Statische Typisierung mittels TypeScript

TypeScript ist keine völlig eigenständige Sprache, sondern eine statische Erweiterung von JavaScript. Während JavaScript die Laufzeitgrundlage bildet, fügt

TypeScript Typinformationen hinzu, die bereits vor der Ausführung vom Compiler geprüft werden.

Zur Veranschaulichung zeigt Listing 7 zunächst eine einfache Addierfunktion in purem JavaScript.

```
1 function add(a, b) {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 console.log(add("Hallo", "Welt"));
```

Listing 7: Eine untypisierte Funktion in JavaScript

Da keine Typen vorgegeben sind, akzeptiert die Funktion beliebige Eingaben. Die Flexibilität ist hoch, die eigentliche fachliche Absicht der Funktion bleibt jedoch im Dunkeln. Listing 8 zeigt im direkten Kontrast, wie TypeScript es erlaubt, diese Absicht unmissverständlich im Code festzuhalten.

```
1 function add(a: number, b: number): number {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 // console.log(add("Hallo", "Welt")); // Führt zu einem Compiler-Fehler
```

Listing 8: Dieselbe Funktion in TypeScript mit expliziten Typen

Durch die expliziten Typannotationen wird die Signatur, also die formale Schnittstelle bestehend aus Funktionsnamen sowie Anzahl, Reihenfolge und Datentypen der Parameter, eindeutig. Ein fehlerhafter Aufruf mit zwei Strings widerspricht diesem Vertrag und wird vom Compiler beanstandet, lange bevor das Programm ausgeführt wird. Der hier verwendete Typ `number` ist dabei der zentrale Baustein für numerische Werte:

Definition 2.1. Der Typ `number`: Im Gegensatz zu Sprachen wie Python, C oder Java, die streng zwischen ganzen Zahlen (`int`) und Kommazahlen (`float`) unterscheiden, fasst TypeScript alle numerischen Werte unter dem Typ `number` zusammen. Programmatisch handelt es sich um 64-Bit-Fließkommazahlen nach dem IEEE-754-Standard [1]. Eine Funktion mit diesem Parameter akzeptiert folglich sowohl `add(3, 4)` als auch `add(3.14, 2.5)`.

2.2.1 Herausforderungen einer naiven Code-Migration

Eine reine Portierung von Python-Code nach TypeScript führt nicht automatisch zu echter Typsicherheit. Da TypeScript als Obermenge von JavaScript konzipiert ist, lässt sich bestehender Python-Code häufig fast unverändert (lediglich mit geänderter Syntax) übernehmen, bleibt im Kern jedoch reines JavaScript. Listing 9 demonstriert eine solche naive Übersetzung.

```
1 function formatUserId(id) {  
2     return "User_" + id;  
3 }  
4 console.log(formatUserId(5));  
5 console.log(formatUserId("admin"));  
6 console.log(formatUserId([1, 2, 3]));
```

Listing 9: Naive Übersetzung einer Funktion nach TypeScript

Die Funktion kompiliert ohne Fehlermeldung, verarbeitet aber völlig unterschiedslos Zahlen, Strings und Arrays. Die Absicht, hier die Formatierung einer numerischen Nutzer-ID, geht komplett verloren. Ein solcher Code ist zwar formal TypeScript, inhaltlich bewahrt er jedoch die Fehleranfälligkeit der dynamischen Programmierung.

Dynamische Typprüfung zur Laufzeit

Auch in untypisierten Umgebungen besitzen Daten zur Laufzeit stets einen konkreten Typ, der sich programmatisch auslesen lässt. In Python wird hierfür typischerweise die Funktion `type()` verwendet (siehe Listing 10), während JavaScript und TypeScript den Operator `typeof` nutzen (siehe Listing 11).

```
1 print(type(42))          # <class 'int'>  
2 print(type("Hallo"))    # <class 'str'>  
3 print(type([1, 2, 3]))  # <class 'list'>
```

Listing 10: Typprüfung zur Laufzeit in Python

```
1 console.log(typeof 42);    // "number"  
2 console.log(typeof "Hallo"); // "string"  
3 console.log(typeof true);  // "boolean"
```

Listing 11: Typprüfung zur Laufzeit in JavaScript und TypeScript

Implikationen fehlender Typisierung (any)

Werden Variablen in TypeScript nicht explizit annotiert und lässt sich ihr Typ nicht eindeutig ableiten (wie bei dem Parameter `id` in der naiven Übersetzung), greift TypeScript auf einen Fallback zurück: Es weist der Variable den Typ `any` zu. Dieser Typ nimmt eine gefährliche Sonderrolle im System ein.

Definition 2.2. Der Typ `any`: Das Schlüsselwort `any` ist der universelle Fluchtweg aus dem Typsystem. Es repräsentiert einen beliebigen JavaScript-Wert ohne jegliche Einschränkungen. Der Compiler erlaubt auf einem `any`-Wert den Aufruf beliebiger Methoden und Eigenschaften, womit die statische Typprüfung für diese Variable faktisch vollständig deaktiviert wird.

Wie Listing 12 verdeutlicht, kann `any` auch explizit vergeben werden, um den Compiler bewusst „blind“ zu machen.

```
1 let value: any = [1, 2, 3];
2 console.log(value.toUpperCase());           // stürzt zur Laufzeit ab
3 console.log(value * 2);                     // ergibt NaN
4 console.log(value.notExistingMethod());    // stürzt zur Laufzeit ab
```

Listing 12: Der Typ `any` hebt die Typprüfung aus

Ein typsicheres Programm müsste validieren, ob `value` überhaupt die Methode `toUpperCase()` besitzt. Bei `any` entfällt diese Kontrolle, was zu Laufzeitfehlern führen kann.

Gefahren der impliziten Typkonvertierung (Koerzision)

Ein weiteres Risiko schlecht typisierten TypeScript-Codes ist die sogenannte *Koerzision* (implizite Typumwandlung). Python verlangt für fast alle Operationen explizite Umwandlungen, wie Listing 13 zeigt.

```
1 values = [1, 2, 3]
2 text = str(values)
3 print(text)           # "[1, 2, 3]"
```

Listing 13: Explizite Umwandlung in Python

JavaScript (und untypisiertes TypeScript) ist an dieser Stelle nachgiebiger und erzwingt oft stillschweigende Umwandlungen, sobald ein Operator dies nahelegt. Listing 14 stellt dieses Verhalten der expliziten Umwandlung gegenüber.

```

1 const values = [1, 2, 3];
2 const implicitText = values + "";    // Koerzision
3 const explicitText = String(values); // Explizite Umwandlung
4 console.log(implicitText);           // "1,2,3"

```

Listing 14: Implizite und explizite Umwandlung in JavaScript

Type Assertions als trügerische Compiler-Anweisung

Neben `any` bietet TypeScript mit der Typbehauptung einen weiteren Mechanismus, der das Typsystem aufweicht und für Einsteiger oft wie eine bequeme Typumwandlung (Type Cast) wirkt. Dies ist jedoch ein Trugschluss.

Definition 2.3. Die Type Assertion (`as`): Das Schlüsselwort `as` ist keine Operation, die zur Laufzeit ausgeführt wird. Es ist eine reine Compiler-Direktive. Sie zwingt den TypeScript-Compiler dazu, einen Wert ab sofort als den angegebenen Typ zu behandeln, lässt den Wert im Arbeitsspeicher aber völlig unangetastet.

Listing 15 verdeutlicht diese trügerische Sicherheit.

```

1 let values: any = [1, 2, 3];
2
3 // Keine echte Laufzeit-Umwandlung, sondern nur eine Behauptung:
4 const text = values as string;
5
6 // Der Wert ist zur Laufzeit weiterhin ein Array!
7 console.log(text.toUpperCase()); // Laufzeitfehler

```

Listing 15: Eine Type Assertion ist keine echte Umwandlung

Das inhaltliche Problem wurde somit nicht gelöst, sondern lediglich vor der Typprüfung versteckt, zur Laufzeit ist die Methode auf dem Array nicht definiert und es kommt zum Absturz.

Methodischer Verzicht auf Typ-Bypässe

Weder `any` noch `as` sind neutrale Komfortfunktionen. Es sind Mechanismen, die die Wirkung des Typsystems gezielt aushebeln. Das erklärte Ziel dieser Studienarbeit war es, den Code nach TypeScript, und nicht bloß nach JavaScript in `.ts`-Dateien, zu überführen. Folglich wurde architektonisch in der gesamten Implementierung auf den Einsatz von `any` und ungesicherten `as`-Behauptungen verzichtet.

2.2.2 Explizite Typannotationen

Der professionelle Gegenentwurf zu Ausweichmechanismen ist die explizite Typannotation. Dabei wird im Programmtext unmissverständlich angegeben, welcher Typ für Variablen, Parameter oder Rückgabewerte zwingend erwartet wird.

Listing 16 zeigt den Unterschied anhand einer Funktion, die aus einer numerischen ID einen formatierten Text generiert.

```
1 // Unpräzise: Lässt völlig unklar, was id sein soll
2 function formatUserIdLoose(id) {
3     return "User_" + id;
4 }
5
6 // Präzise: Zwingender Vertrag durch explizite Annotationen
7 function formatUserIdStrict(id: number): string {
8     return "User_" + id;
9 }
```

Listing 16: Unpräzise vs. präzise formulierte Funktion in TypeScript

Die strikte Variante formuliert einen eindeutigen Vertrag, der keinen Interpretationsspielraum zulässt. Die Signatur verlangt nicht nur eine `number` als Eingabe, sondern garantiert auch einen spezifischen Rückgabebetyp, in diesem Fall einen Text:

Definition 2.4. Der Typ `string`: Dieser primitive Typ repräsentiert jegliche Form von Textdaten. Er besteht aus einer unveränderlichen (immutablen) Sequenz von 16-Bit-Werten nach dem UTF-16-Standard. Einmal im Speicher angelegt, kann ein String nicht modifiziert werden; Operationen wie String-Verkettungen erzeugen stets eine neue Zeichenkette.

Atomare und strukturierte Basistypen

Neben den bereits eingeführten primitiven Werten wie Zahlen und Texten, ist ein dritter atomarer Baustein essenziell für die Steuerung des Programmflusses:

Definition 2.5. Der Typ `boolean`: Er repräsentiert logische Wahrheitswerte und kann ausschließlich die Zustände `true` oder `false` annehmen. Er entspricht exakt dem `bool`-Typ aus Python und ist essenziell für Kontrollfluss-Bedingungen.

Zusätzlich zu diesen atomaren Bausteinen erlaubt TypeScript die Definition zusammengesetzter Strukturen. Wie Listing 17 demonstriert, können Arrays oder spezifisch geformte Objekte direkt annotiert werden.

```

1 // Atomare Typen
2 let username: string = "alice";
3 let loginCount: number = 5;
4 let isActive: boolean = true;
5
6 // Zusammengesetzte Typen
7 let roles: string[] = ["admin", "staff"];
8 let userProfile: { id: number; name: string } = {
9     id: 42,
10    name: "Alice"
11 };

```

Listing 17: Atomare und zusammengesetzte TypeScript-Typen

Bereits die simple Typdefinition von `userProfile` legt eine klare Struktur mit zwei benannten und typisierten Bestandteilen fest. Die Fähigkeit, komplexe Verträge im Code zu verankern, ist Grundlage sicherer Architekturen.

Eingrenzung von Wertebereichen durch Literal und Union Types

Oft reicht es nicht aus, einen Wert pauschal als `string` zu deklarieren. Wenn fachlich nur eine feste Menge spezifischer Ausprägungen zulässig ist (etwa bestimmte Nutzerrollen), bietet TypeScript zwei eng verwobene Konzepte:

Definition 2.6. Literal Types: Ein Literal Type beschreibt nicht eine ganze Menge von Werten, sondern repräsentiert exakt einen einzigen, konkreten Wert (z. B. exakt die Zeichenkette `"admin"`).

Definition 2.7. Union Types (I): Ein Union Type kombiniert mehrere Typen durch ein logisches „Oder“. Eine Variable dieses Typs darf zur Laufzeit jeden der verknüpften Typen annehmen.

Ihre immense Flexibilität gewinnen Literal Types gerade durch diese Kombination mit Union Types. Listing 18 zeigt, wie sich Zustandsräume dadurch präzise eingrenzen lassen.

Dieses Konzept transformiert lose Text-Konventionen in maschinenprüfbare Verträge. Python unterstützt dies ebenfalls sehr ähnlich und nutzt dafür `Literal` aus dem `typing`-Modul, wie Listing 19 belegt.

Modellierung fehlender Werte (`null` und `undefined`)

In realen Programmen tritt häufig der Fall auf, dass ein gesuchter Wert schlicht nicht vorhanden ist. Ein präzises Typsystem muss die Möglichkeit eines solchen fehlenden Wertes ausdrücklich modellieren, anstatt einfach darauf zu hoffen,

```

1 type Role = "admin" | "user" | "guest";
2
3 let currentRole: Role = "user";
4 currentRole = "admin";
5 // currentRole = "superuser";
6 // Compiler-Fehler: Typ '"superuser"' ist nicht zuweisbar

```

Listing 18: Union Type aus Literal Types in TypeScript

```

1 from typing import Literal
2
3 Role = Literal["admin", "user", "guest"]
4
5 def is_privileged(role: Role) -> bool:
6     return role == "admin"

```

Listing 19: Literal Types in Python

dass der aufrufende Code damit zurechtkommt. TypeScript verwendet dafür zwei spezifische Typen, die verschiedene Nuancen der Abwesenheit ausdrücken:

Definition 2.8. Die Typen `undefined` und `null`: Der Typ `undefined` bedeutet typischerweise, dass ein Wert noch nicht initialisiert wurde oder in einem Suchvorgang nicht gefunden werden konnte. Der Typ `null` wird hingegen meist genutzt, um die absichtliche, bewusste Abwesenheit eines Wertes durch den Entwickler zu markieren. Beide Typen repräsentieren zur Laufzeit einen fehlenden Wert, haben aber eine leicht unterschiedliche Semantik.

Die Suche nach einer E-Mail-Adresse in einer Datenbank, wie in Listing 20 dargestellt, verdeutlicht dieses Prinzip.

```

1 function findEmail(username: string): string | undefined {
2     if (username === "admin") {
3         return "admin@example.com";
4     }
5     return undefined;
6 }

```

Listing 20: Ein möglicher fehlender Rückgabewert in TypeScript

Da die Funktion nicht zwingend ein Ergebnis findet, reicht der Rückgabebetyp `string` allein nicht aus. Die Signatur `string | undefined` nutzt einen Union Type und

hält somit ausdrücklich fest, dass entweder ein String oder eben kein nutzbarer Wert zurückkommt.

Python modelliert dieses Problem auf der Ebene der Laufzeitwerte ganz ähnlich, wie Listing 21 zeigt. Dort wird ein potenziell fehlender Wert typischerweise durch `None` ausgedrückt.

```
1 def find_email(username):  
2     if username == "admin":  
3         return "admin@example.com"  
4     return None
```

Listing 21: Möglicher fehlender Rückgabewert in Python

Der Grundgedanke ist in beiden Sprachen identisch: Ein Rückgabewert kann vorhanden sein oder fehlen. Entscheidend ist jedoch, wie streng diese Möglichkeit anschließend durch den Compiler der jeweiligen Sprache eingefordert wird.

Strikte Nullwert-Prüfung (`strictNullChecks`)

Die bloße Existenz von `null` und `undefined` erzeugt noch keine automatische Typsicherheit. Ob der Compiler diese Fälle als eigenständige Problemquellen ernst nimmt, steuert in TypeScript eine spezifische Konfigurationseinstellung:

Definition 2.9. Die Compiler-Option `strictNullChecks`: Ist diese Option deaktiviert, behandelt TypeScript fehlende Werte nachsichtig, `null` und `undefined` dürfen überall dort übergeben werden, wo eigentlich beispielsweise ein `string` erwartet wird. Ist sie jedoch aktiviert, trennt der Compiler diese Typen strikt von regulären Werten. Ein potenziell fehlender Wert muss dann zwingend im Code auf seine Existenz geprüft werden, bevor auf ihn zugegriffen werden darf.

Führt man das vorherige E-Mail-Beispiel fort, wird die Tragweite dieser Option in Listing 22 sofort sichtbar.

```
1 const email = findEmail("guest");  
2 console.log(email.toUpperCase()); // Compiler-Fehler bei aktivem strictNullChecks
```

Listing 22: Fehlerhafte Nutzung ohne vorherige Prüfung

Diese Zeile ist hochgradig fehleranfällig. Liefert die Suche `undefined`, schlägt der Aufruf von `toUpperCase()` zur Laufzeit fehl. Mit aktiviertem `strictNullChecks` macht der TypeScript-Compiler diesen Widerspruch schon während des Tippens im Editor sichtbar und verweigert die Übersetzung.

Die saubere Lösung besteht darin, den fehlenden Wert im Kontrollfluss explizit zu behandeln, wie Listing 23 demonstriert.

```
1 const email = findEmail("guest");
2 if (email !== undefined) {
3     console.log(email.toUpperCase()); // email kann nur string sein
4 }
```

Listing 23: Explizite Behandlung eines möglichen undefined-Werts

Einer der häufigsten Fehler in Programmen besteht darin, blind davon auszugehen, dass ein Wert vorhanden ist. Für die Notebooks war `strictNullChecks` deshalb keine kosmetische Zusatzregel, sondern eine elementare Sicherheitsmaßnahme, welche über einen Patch am Notebookkernel eingebaut wurde.

Der Typ `unknown` als sichere Abstraktion

Neben `null` und `undefined` bietet TypeScript einen weiteren wichtigen Spezialfall an, um mit Unsicherheit umzugehen. Dieser ist besonders dann nützlich, wenn zwar feststeht, dass ein Wert existiert, dessen genauer Typ aber noch nicht bekannt ist (beispielsweise bei externen Eingaben).

Definition 2.10. Der Typ `unknown`: Dieser Typ ist das typsichere Gegenstück zu `any`. Während `any` die Typprüfung faktisch deaktiviert, verbietet `unknown` zunächst jegliche Interaktion. Der Compiler zwingt den Entwickler dazu, den Wert durch Laufzeitprüfungen präzise einzuengen, bevor typspezifische Methoden aufgerufen werden dürfen.

Listing 24 zeigt, wie der Compiler unsichere Operationen auf einem `unknown`-Wert rigoros blockiert.

```
1 let value: unknown = [1, 2, 3];
2
3 // Nicht erlaubt, solange der Typ noch unbekannt ist:
4 // console.log(value.toUpperCase()); // Compiler-Fehler
```

Listing 24: Der Typ `unknown` erzwingt eine vorherige Prüfung

Erst nach einer erfolgreichen Typprüfung im Kontrollfluss schaltet der Compiler die typspezifischen Methoden frei, was in Listing 25 ersichtlich wird.

Auf diese Weise lassen sich unsichere Daten sicher modellieren, ohne den Type Checker auszuhebeln.

```
1 let value: unknown = "Alice";
2
3 if (typeof value === "string") {
4     // Innerhalb dieses Blocks weiß der Compiler, dass value ein string ist.
5     console.log(value.toUpperCase());
6 }
```

Listing 25: Einengung von `unknown` durch Typprüfung im Kontrollfluss

Vergleich mit statischer Analyse in Python (mypy)

Auf den ersten Blick besitzen Python und TypeScript ähnliche Ausdrucksmittel für Typen. Wie Listing 26 zeigt, lassen sich auch in Python Variablen, Parameter und Rückgabewerte mittels Type Hints annotieren.

```
1 def format_user_id(user_id: int) -> str:
2     return f"User_{user_id}"
```

Listing 26: Type Hints in Python

Ein fundamentaler Unterschied besteht jedoch in der Durchsetzung: Python ignoriert diese Type Hints zur Laufzeit vollständig. Ein fehlerhafter Aufruf mit falschen Datentypen führt nicht zwingend zu einem sofortigen Fehler. Um die Typinformationen systematisch zu prüfen, wird ein externes Werkzeug wie *mypy* benötigt, das den Code statisch analysiert.

Zudem fehlt Python ein strenges Äquivalent zum sicheren `unknown`. Der Typ `Any` in Python entspricht vielmehr dem unsicheren `any` in TypeScript. Listing 27 stellt diese Konzepte direkt gegenüber.

2.2.3 Strukturelles Typing und Interface-Verträge

Neben primitiven Werten erfordert die Modellierung von Software verlässliche Verträge zwischen Komponenten. Ein solcher Vertrag legt fest, welche Eigenschaften oder Methoden ein Wert besitzen muss, um genutzt werden zu können. TypeScript verwendet dafür unter anderem *Interfaces*.

Entscheidend ist dabei ein Konzept, das TypeScript grundlegend von Sprachen wie Java oder C# unterscheidet:

```

1 from typing import Any
2
3 def normalize(value: Any) -> str:
4     return str(value) # mypy erlaubt hier völlig beliebige Eingaben

```

```

1 function normalize(value: unknown): string {
2     if (typeof value === "string") {
3         return value;
4     }
5     return String(value); // TypeScript erzwingt die Prüfung oder Konvertierung
6 }

```

Listing 27: Python Any vs. TypeScript unknown

Definition 2.11. Strukturelles Typing (Statisches Duck-Typing): TypeScript prüft Typkompatibilität nicht anhand einer expliziten Klassenhierarchie (Nominales Typing), sondern rein anhand der „Gestalt“ (Struktur) eines Objekts. Besitzt ein Objekt alle Eigenschaften und Methoden, die ein Interface verlangt, gilt es als kompatibel zu diesem Interface, unabhängig davon, ob es explizit davon erbt oder nicht.

Listing 28 definiert einen solchen strukturellen Vertrag in Form des Interfaces `Displayable`.

```

1 interface Displayable {
2     getLabel(): string;
3     isActive(): boolean;
4 }

```

Listing 28: Ein Interface als Vertrag über benötigte Methoden

Dieses Interface fordert lediglich das Vorhandensein zweier spezifischer Methoden. Listing 29 demonstriert, dass jede Klasse, die diese Signatur zufällig oder absichtlich erfüllt, automatisch typkompatibel ist.

Dieser strukturelle Ansatz entkoppelt Funktionalität von starren Klassenhierarchien. Datenstrukturen können polymorph mit völlig unterschiedlichen Elementen arbeiten, solange diese den vertraglich zugesicherten Methodenapparat bereitstellen.

```

1 class UserAccount {
2     constructor(public username: string, public active: boolean) {}
3
4     getLabel(): string {
5         return this.username;
6     }
7
8     isActive(): boolean {
9         return this.active;
10    }
11 }
12
13 // Zuweisung ist zulässig, da Struktur des Objekts mit Interface übereinstimmt:
14 const item: Displayable = new UserAccount("alice", true);

```

Listing 29: Eine Klasse erfüllt ein Interface über ihre reine Struktur

Äquivalente Strukturierung in Python (Protocol und TypedDict)

Python unterstützt strukturelles Denken zur Laufzeit nativ durch sein dynamisches Duck-Typing. Um dieses Verhalten jedoch statisch überprüfbar zu machen, bietet das `typing`-Modul (in Kombination mit `mypy`) spezifische Hilfsmittel an.

Analog zu einem TypeScript-Interface ermöglicht die Klasse `Protocol` die Definition von Methoden-Verträgen. Listing 30 zeigt, wie ein solcher statischer Vertrag in Python formuliert wird.

```

1 from typing import Protocol
2
3 class Displayable(Protocol):
4     def get_label(self) -> str: ...
5     def is_active(self) -> bool: ...

```

Listing 30: Ein Protocol in Python als statischer Vertrag

Klassen müssen auch in Python nicht explizit von diesem Protocol erben, um als kompatibel zu gelten; die bloße Erfüllung der Signatur reicht aus. Während strukturelle Kompatibilität in TypeScript jedoch das fundamentale Designprinzip bildet, bleibt sie in Python ein optionales Konstrukt.

Für reine Datenobjekte (ohne eigenes Verhalten) bietet Python zusätzlich `TypedDict`. Wie Listing 31 demonstriert, lassen sich damit Dictionary-Strukturen mit festen Schlüsseln und Typen definieren.

```
1 from typing import TypedDict
2
3 class Address(TypedDict):
4     street: str
5     zip_code: int
6     city: str
```

Listing 31: Feste Datenstruktur mit TypedDict in Python

Interfaces vs. Type Aliase in TypeScript

In TypeScript gibt es für die Modellierung von Datenstrukturen gleich zwei Werkzeuge, die auf den ersten Blick identisch wirken: `interface` und `type`. Während `interface` (wie in Abschnitt 2.2.3 gezeigt) historisch aus der objektorientierten Welt stammt und primär die Gestalt von Objekten oder Klassen beschreibt, ist `type` ein deutlich universelleres Konzept:

Definition 2.12. Type Alias (`type`): Ein Type Alias gibt einem beliebigen Typ einen neuen, aussagekräftigen Namen. Im Gegensatz zu einem `interface` kann ein `type` nicht nur Objektstrukturen, sondern auch die bereits eingeführten Union Types, Tupel oder primitive Werte benennen.

Listing 32 stellt die TypeScript-Äquivalenz zum Python-`TypedDict` dar. Es zeigt, wie fließend `interface` und `type` für reine Datenobjekte ineinander übergehen, während `type` bei komplexeren Kombinationen (wie Union Types) alternativlos ist.

Für die Architektur der portierten Software bedeutet dies eine klare Arbeitsteilung: Sobald Verträge für Klassen oder polymorphes Verhalten (Methoden) definiert werden, greift die Codebasis bevorzugt auf `interface` zurück. Geht es jedoch um die Definition von reinen Datencontainern, Tupeln oder die Kombination von Typen (wie bei `UnsafeMail`), ist der `type`-Alias das Werkzeug der Wahl. Diese saubere semantische Trennung verbessert die Lesbarkeit und macht die TypeScript-Notebooks in ihrer Typbeschreibung ausdrucksstärker als die Python-Vorlage.

Generische Programmierung im Sprachvergleich

Um Algorithmen und Datenstrukturen typsicher, aber dennoch hochgradig wiederverwendbar zu gestalten, bedarf es einer weiteren Abstraktionsebene:

```

1 // Variante 1: Interface (Fokus auf die Objektform, oft nachträglich erweiterbar)
2 interface Address {
3     street: string;
4     zipCode: number;
5     city: string;
6 }
7
8 // Variante 2: Type Alias (Fokus auf einen festen Namen für eine feste Struktur)
9 type UserData = {
10     name: string;
11     email: string;
12     address: Address; // Nahtlose Kombination von Type und Interface
13 };
14
15 // Die exklusive Stärke von 'type': Benennung von Unions oder Tupeln
16 // (Dies wäre mit einem 'interface' syntaktisch nicht möglich)
17 type UnsafeMail = string | undefined;

```

Listing 32: Objektstrukturen und Aliase: interface und type in TypeScript

Definition 2.13. Generische Programmierung (Generics): Generics fungieren als Variablen für Typen. Anstatt eine Funktion starr auf einen Typen festzulegen oder auf das unsichere `any` auszuweichen, wird ein Typ-Parameter (meist `T`) definiert. Dieser Platzhalter wird erst zum Zeitpunkt des tatsächlichen Aufrufs durch den konkreten Datentyp ersetzt.

Listing 33 demonstriert den Einsatz einer solchen generischen Funktion in TypeScript.

```

1 function first<T>(items: T[]): T | undefined {
2     if (items.length === 0) return undefined;
3     return items[0];
4 }
5
6 // firstNumber ist number | undefined
7 const firstNumber = first<number>([10, 20, 30]);
8 // firstName ist string | undefined
9 const firstName = first<string>(["Alice", "Bob"]);

```

Listing 33: Eine generische Funktion in TypeScript

Dasselbe Konzept wird in Python mittels `TypeVar` umgesetzt, wie der direkte Vergleich in Listing 34 zeigt.

Die wahre architektonische Stärke von Generics entfaltet sich in der Kombi-

```

1 from typing import TypeVar
2
3 T = TypeVar("T")
4
5 def first(items: list[T]) -> T | None:
6     if len(items) == 0:
7         return None
8     return items[0]

```

Listing 34: Eine generische Funktion in Python

nation mit strukturellen Schranken (Type Constraints). Listing 35 veranschaulicht, wie sich mit dem Schlüsselwort `extends` (in TypeScript) oder dem Parameter `bound` (in Python) erzwingen lässt, dass der variabel eingesetzte Typ einen bestimmten Basis-Vertrag erfüllen muss.

```

1 // TypeScript: T muss mindestens die Methoden aus 'Displayable' besitzen
2 function filterActive<T extends Displayable>(items: T[]): T[] {
3     return items.filter(item => item.isActive());
4 }

```

```

1 # Python: S wird an das Protocol 'Displayable' gebunden
2 S = TypeVar("S", bound=Displayable)
3
4 def filter_active(items: list[S]) -> list[S]:
5     return [item for item in items if item.is_active()]

```

Listing 35: Generics mit strukturellen Schranken im Vergleich

Kontrollflussbasierte Typverfeinerung (Type Narrowing)

Sobald ein Wert als Union Type definiert ist, verlangt der Compiler Klarheit darüber, welcher Fall zur Laufzeit konkret vorliegt, bevor typspezifische Operationen zugelassen werden.

Definition 2.14. Type Narrowing: Dies ist der Prozess der schrittweisen Typ-Einengung. Der TypeScript-Compiler analysiert den Kontrollfluss des Codes (wie `if`-Verzweigungen) in Kombination mit Laufzeitprüfungen. Kann bewiesen werden, dass eine Variable einen bestimmten Typ hat, behandelt der Compiler sie innerhalb dieses Blocks vollautomatisch als diesen engeren Typ.

Listing 36 verdeutlicht, wie diese statische Analyse in der Praxis auf Basis des

`typeof`-Operators funktioniert.

```
1 function formatValue(value: string | number): string {
2     if (typeof value === "string") {
3         // Narrowing greift: value ist hier zwingend ein string
4         return value.toUpperCase();
5     }
6     // Durch den frühen Return oben MUSS value hier eine number sein
7     return value.toFixed(2);
8 }
```

Listing 36: Type Narrowing durch Kontrollfluss und `typeof`

Während `typeof` für primitive Datentypen geeignet ist, liefert es bei Instanzen komplexer Klassen pauschal nur `"object"`. Für die Überprüfung von Objekten bietet sich daher der Operator `instanceof` an. Listing 37 demonstriert diese klassenbasierte Verfeinerung.

```
1 class UserAccount {
2     constructor(public username: string, public active: boolean) {}
3 }
4
5 function describe(user: UserAccount | string): string {
6     if (user instanceof UserAccount) {
7         return `Konto: ${user.username} (Aktiv: ${user.active})`;
8     }
9     return `Gast: ${user}`;
10 }
```

Listing 37: Klassenprüfung mit `instanceof`

Vollständigkeitsprüfung mittels `never`-Typ

Das Type Narrowing wirft unweigerlich eine logische Folgefrage auf: Welchen Typ hat ein Wert, wenn alle theoretisch möglichen Fälle einer Union vollständig abgehandelt wurden? In TypeScript nimmt dieser logisch un erreichbare Zustand einen besonderen Typ an:

Definition 2.15. Der Typ `never`: Dieser Typ repräsentiert einen Wert, der niemals existieren darf und Code, der niemals erreicht werden sollte. Er bildet den tiefsten Punkt in der Typhierarchie. Nichts ist `never` zuweisbar.

`never` wird in der Praxis als genialer Trick für das sogenannte *Exhaustiveness Checking* (Vollständigkeitsprüfung) eingesetzt. Listing 38 zeigt, wie dieser Typ

garantiert, dass Erweiterungen des Datenmodells (z. B. ein neuer Bestellstatus im Online-Shop) vom Entwickler niemals unbemerkt übersehen werden können.

```
1 type OrderStatus = "pending" | "shipped" | "delivered";
2
3 function translateStatus(status: OrderStatus): string {
4     switch (status) {
5         case "pending": return "In Bearbeitung";
6         case "shipped": return "Versendet";
7         case "delivered": return "Zugestellt";
8
9         default: {
10             // Compile-Fehler, falls OrderStatus in Zukunft z.B. um "canceled"
11             // erweitert wird, ohne diesen Switch-Block anzupassen:
12             const exhaustiveCheck: never = status;
13             return exhaustiveCheck;
14         }
15     }
16 }
```

Listing 38: Vollständige Fallunterscheidung mit `never`

2.3 Fazit zur architektonischen Neuausrichtung

Die Herausforderung der ursprünglichen Architektur lag nicht in einer grundsätzlichen Unzulänglichkeit von *mypy* als Werkzeug. Für klassische, dateibasierte Python-Projekte ist *mypy* ein leistungsfähiger statischer Checker. Die Schwierigkeiten resultierten vielmehr aus der Diskrepanz zwischen einer nachträglich aufgesetzten statischen Typprüfung und der hochdynamischen, zellenbasierten Laufzeitumgebung von Jupyter-Notebooks.

In der Python-Umgebung musste die Typprüfung durch externe Werkzeuge wie *nb-mypy* nachgerüstet werden. Im praktischen Lehrbetrieb offenbarten sich dabei zwei gravierende architektonische Reibungspunkte:

- **Fehlendes Type Narrowing bei `match-case`:** Bei der Verwendung moderner Sprachkonstrukte wie dem strukturellen Pattern Matching (`match` und `case`) stieß die Notebook-Integration des Checkers an ihre Grenzen. Selbst wenn alle theoretisch möglichen Fälle logisch vollständig abgehandelt waren, erkannte das Tool diese Vollständigkeit (Exhaustiveness) nicht zuverlässig. Um Fehlermeldungen über angeblich fehlende Rückgabewerte zu vermeiden, mussten gezwungenermaßen toter Code geschrieben (z. B. ein redundantes `return None`) oder legitime Konstrukte per `# type: ignore` unterdrückt werden.

- **Zustandsverlust bei `%run`-Befehlen:** Um Code aus Basis-Notebooks in anderen Notebooks wiederzuverwenden, macht die interaktive Umgebung Gebrauch von Magic Commands wie `%run`. Dies korrumpierte jedoch regelmäßig den internen Typ-Zustand von *nb-mypy*. Als Workaround musste die Typprüfungs-Erweiterung manuell entladen werden, was ihren inhärenten Zweck ad absurdum führte.

Die Neuausrichtung des Projekts behält die didaktischen Vorteile der interaktiven Arbeit bei, wechselt jedoch die technologische Basis, um genau diese Brüche zu heilen. Durch den Einsatz von TypeScript in Kombination mit dem *ts/ab*-Kernel wird die Typprüfung zu einem integralen Bestandteil der Ausführungsumgebung. Der TypeScript-Kernel verifiziert den Code durch seine native Kontrollflussanalyse direkt während der Eingabe und über Zellengrenzen hinweg, ohne auf fehleranfällige Zusatzskripte oder ständige Neustarts der Prüfwerkzeuge angewiesen zu sein.

3 RecursiveSet: API und fachliche Modellierung

Wie bereits in Kapitel 1 anhand von Listing 1 und Listing 2 zu sehen war, verhalten sich Mengen in Python und TypeScript unterschiedlich. Dieses Kapitel erklärt zunächst, worauf diese Unterschiede fachlich und technisch zurückzuführen sind. Anschließend wird gezeigt, wie die Bibliothek `RecursiveSet` diese Unterschiede für die Arbeit mit mathematischen Mengenstrukturen ausgleicht und dadurch eine an Python angelehnte Arbeitsweise in TypeScript ermöglicht.

3.1 Mathematische Mengen und Wohlunterscheidbarkeit

Was ist überhaupt eine Menge im mathematischen Sinn? Georg Cantor beschreibt eine Menge als „Zusammenfassung bestimmter wohlunterschiedener Objekte [...] zu einem Ganzen“ [2, S. 481]. Für die Umsetzung mengenbasierter Algorithmen in einer Programmiersprache ergibt sich daraus unmittelbar eine praktische Frage: Wann gelten zwei Elemente als gleich?

Python und TypeScript beziehungsweise JavaScript beantworten diese Frage unterschiedlich. In Python können unveränderliche, hashbare Werte wie Tupel als Elemente einer Menge verwendet werden. Zwei Tupel gelten dabei als gleich, wenn sie denselben Inhalt haben (Wertsemantik). In JavaScript werden Objekte wie Arrays in einem `Set` dagegen über ihre Referenzidentität unterschieden. Zwei Arrays mit gleichem Inhalt gelten also nur dann als gleich, wenn es sich um dieselbe physische Array-Instanz im Arbeitsspeicher handelt.

Listing 39 zeigt zunächst das Verhalten in Python:

```
1 # Python: Tupel sind unveränderlich und als Set-Elemente geeignet
2 zahlen_menge: set[tuple[int, int]] = set()
3
4 zahlen_menge.add((1, 2))
5 zahlen_menge.add((2, 1))
6
7 print(zahlen_menge)           # {(1, 2), (2, 1)}
8 print((2, 1) in zahlen_menge) # True
```

Listing 39: Mengenverhalten in Python

Die Mitgliedschaftsabfrage funktioniert hier, weil Tupel unveränderlich sind und in Mengen anhand ihrer inhaltlichen Gleichheit berücksichtigt werden.

Anders verhält es sich in TypeScript, wie Listing 40 demonstriert. Dieselbe Modellierungsidee führt zu einem anderen Ergebnis:

```
1 type ZahlenTupel = [number, number];
2 const zahlenMenge = new Set<ZahlenTupel>();
3
4 zahlenMenge.add([1, 2]);
5 zahlenMenge.add([2, 1]);
6
7 // Die Suche nach einem inhaltlich gleichen, aber neu erzeugten Array:
8 console.log(zahlenMenge.has([2, 1])); // false
```

Listing 40: Mengenverhalten in TypeScript

Hier schlägt die Abfrage fehl, weil das neu erzeugte Array `[2, 1]` in Zeile 7 eine andere Speicheradresse besitzt als das in Zeile 5 eingefügte Array.

Für Algorithmen aus der diskreten Mathematik und theoretischen Informatik ist dieser Unterschied essenziell. Da TypeScript native inhaltsbasierte Vergleiche für strukturierte Objekte nicht unterstützt, ist für deren Verarbeitung eine eigene Datenstruktur erforderlich.

3.2 Kernbausteine der Bibliothek

Um die referenzbasierte Gleichheitsprüfung der JavaScript-Laufzeitumgebung zu umgehen, stellt die Bibliothek drei Datenstrukturen bereit. Diese Bausteine garantieren die in der Mathematik vorausgesetzte Wertsemantik und bilden das Fundament, um Zustände, Übergänge und Alphabet-Zuordnungen abbilden zu können.

3.2.1 Die Datenstruktur `RecursiveSet`

Die bisherigen Beispiele haben gezeigt, dass native Mengen in TypeScript für strukturelle mathematische Modelle nur eingeschränkt geeignet sind. An dieser Stelle setzt `RecursiveSet` an. Auf API-Ebene orientiert sie sich an der aus Python vertrauten Arbeit mit Mengen: Elemente können hinzugefügt, entfernt und auf Mitgliedschaft geprüft werden. Für die Benutzung verhält sich `RecursiveSet` wie eine gewöhnliche Mengenstruktur, während sie intern die Wertsemantik sicherstellt.

Hinzufügen von Elementen

Das Einfügen neuer Elemente erfolgt in beiden Sprachen über die `add`-Methode. Listing 41 zeigt, dass sich die Benutzung deckt.

Mitgliedschaftstest

Die Prüfung auf Mitgliedschaft gehört zu den Grundoperationen. Während Python hierfür den Operator `in` verwendet, stellt `RecursiveSet` die Methode `has()` bereit

```

1 alphabet = {"a", "b", "c"}
2 alphabet.add("d")
3
4 print(alphabet)  # {'a', 'b', 'c', 'd'}

```

```

1 const alphabet = new RecursiveSet<string>("a", "b", "c");
2 alphabet.add("d");
3
4 console.log(alphabet); // {a, b, c, d}

```

Listing 41: Hinzufügen von Elementen in Python und mit RecursiveSet

(siehe Listing 42).

```

1 alphabet = {"a", "b", "c"}
2
3 print("b" in alphabet)  # True
4 print("x" in alphabet)  # False

```

```

1 const alphabet = new RecursiveSet<string>("a", "b", "c");
2
3 console.log(alphabet.has("b")); // true
4 console.log(alphabet.has("x")); // false

```

Listing 42: Mitgliedschaftstest in Python und mit RecursiveSet

Entfernen von Elementen

Schließlich müssen Elemente auch wieder aus einer Menge entfernt werden können. Listing 43 zeigt die Anwendung beider Ansätze.

Die zentrale Stärke der Bibliothek zeigt sich jedoch erst dann, wenn nicht nur primitive Strings, sondern strukturierte und rekursive Werte als Elemente betrachtet werden.

3.2.2 Strukturierte Elemente: Die Tuple-Klasse

Neben Mengen stützen sich theoretische Modelle häufig auf Tupel. Ein Tupel ist eine endliche, geordnete Sequenz von Elementen. Im Unterschied zu Mengen ist die Reihenfolge der Einträge dabei strikt, und Mehrfachvorkommen sind zulässig. Ein Beispiel ist ein Koordinatenpaar (x, y) in einem zweidimensionalen Raum. Für zwei Punkte $P = (2, 5)$ und $Q = (5, 2)$ gilt daher mathematisch $P \neq Q$.

```

1 alphabet = {"a", "b", "c"}
2 alphabet.remove("b")
3 print(alphabet) # {'a', 'c'}

```

```

1 const alphabet = new RecursiveSet<string>("a", "b", "c");
2 alphabet.remove("b");
3 console.log(alphabet); // {a, c}

```

Listing 43: Entfernen von Elementen in Python und mit RecursiveSet

Da Python-Tupel unveränderliche Sequenzen sind, eignen sie sich sehr gut zur Modellierung solcher Datenstrukturen. Wird ein solches Modell in TypeScript mit Arrays nachgebildet, entstehen die in Abschnitt 3.1 beschriebenen Referenz-Probleme.

Die Bibliothek begegnet diesem Sachverhalt mit der `Tuple`-Klasse. Sie dient als geordneter, unveränderlicher Container für strukturierte Werte. Listing 44 zeigt dies an einem Beispiel aus der Modellierung besuchter Roboter-Positionen.

```

1 visited_coords: set[tuple[int, int]] = set()
2 visited_coords.add((2, 5))
3
4 print((2, 5) in visited_coords) # True

```

```

1 const visitedCoords = new RecursiveSet<Tuple<[number, number]>>();
2 visitedCoords.add(new Tuple(2, 5));
3
4 const searchPoint = new Tuple(2, 5);
5 console.log(visitedCoords.has(searchPoint)); // true

```

Listing 44: Roboter-Koordinaten: Tuplel in Python vs. TypeScript

3.2.3 Assoziative Daten: Die `RecursiveMap`

Sobald Koordinaten oder Zustände modelliert sind, müssen diesen in vielen Algorithmen Werte zugeordnet werden. In der Informatik übernimmt diese Aufgabe eine Map (Dictionary). Mathematisch entspricht dies einer Abbildung $f: \text{Schlüssel} \rightarrow \text{Wert}$.

Werden strukturierte Werte als Schlüssel verwendet, muss der Abruf über ihren inhaltlichen Aufbau erfolgen. Die Klasse `RecursiveMap<K, V>` stellt hierfür eine asso-

ziative Datenstruktur bereit, deren Schlüssel dieselbe Wertsemantik verwenden wie `RecursiveSet`.

Listing 45 illustriert dies an einem einfachen Rastermodell.

```
1 grid: dict[tuple[int, int], str] = {}
2 grid[(2, 5)] = "Queen"
3
4 print(grid[(2, 5)]) # "Queen"
```

```
1 const safeGrid = new RecursiveMap<Tuple<[number, number]>, string>();
2 safeGrid.set(new Tuple(2, 5), "Queen");
3
4 const foundPiece = safeGrid.get(new Tuple(2, 5));
5 console.log(foundPiece); // "Queen"
```

Listing 45: Dictionaries in Python vs. `RecursiveMap` in TypeScript

3.3 Verarbeitung und Lebenszyklus strukturierter Daten

Mit den Bausteinen zur Speicherung von Werten stellt sich die Frage nach deren Manipulation. Theoretische Algorithmen erfordern es, Mengen zu kombinieren, funktional zu transformieren und ineinander zu verschachteln. Die folgenden Abschnitte beschreiben den Lebenszyklus dieser Strukturen sowie die API zur Durchführung von Mengenoperationen.

3.3.1 Der Freeze-Contract: Immutabilität sichern

Sobald Mengen selbst wieder in größere Strukturen eingebettet werden (etwa bei Potenzmengen), darf ein Element seinen inhaltlichen Zustand nachträglich nicht mehr verändern, da dies die interne Hashtabelle der Elternmenge korrumpieren würde.

In Python ist dies auf Typ-Ebene gelöst: Ein veränderliches `set` wirft einen `TypeError`, wenn man versucht, es in ein anderes Set einzufügen. Es muss manuell in ein unveränderliches `frozenset` konvertiert werden.

In TypeScript greift an dieser Stelle der sogenannte *Freeze-Contract*. Er legt fest, dass ein `RecursiveSet` ab dem Moment seiner Einbettung automatisch als logisch fixiert (frozen) gilt. Ein expliziter Typwechsel entfällt.

Listing 46 zeigt dieses Verhalten im direkten Sprachvergleich.

Soll ein fixierter Wert dennoch als Ausgangspunkt für weitere Berechnungen dienen, muss eine veränderliche Kopie erzeugt werden. Die Bibliothek stellt hierfür

```

1 inner_set = {1, 2}
2 outer_set = set()
3
4 # outer_set.add(inner_set) würde einen TypeError werfen
5 outer_set.add(frozenset(inner_set))

```

```

1 const innerSet = new RecursiveSet<number>(1, 2);
2 const outerSet = new RecursiveSet<RecursiveSet<number>>();
3
4 outerSet.add(innerSet); // innerSet wird hier automatisch eingefroren
5
6 // innerSet.add(3); // Wirft zur Laufzeit einen Fehler (Frozen Set modified)

```

Listing 46: Der Freeze-Contract bei geschachtelten Mengen

`mutableCopy()` beziehungsweise `clone()` bereit (siehe Listing 47).

```

1 const workingCopy = innerSet.mutableCopy(); // alternativ: innerSet.clone()
2 workingCopy.add(3);
3
4 console.log(innerSet); // {1, 2} (Original bleibt fixiert)
5 console.log(workingCopy); // {1, 2, 3}

```

Listing 47: Arbeiten mit einer veränderlichen Kopie

3.3.2 Klassische Mengenoperationen

Neben der Konstruktion einzelner Mengen spielen klassische Mengenoperationen wie Vereinigung, Schnittmenge, Differenz und symmetrische Differenz eine Rolle. Während Python diese mathematischen Operationen als direkte Operatoren anbietet, wird dieselbe Logik in `RecursiveSet` über API-Aufrufe formuliert.

Vereinigung mit `union`

Die Vereinigung zweier Mengen enthält alle Elemente, die in mindestens einer der beiden Ausgangsmengen vorkommen ($A \cup B = \{x \mid x \in A \vee x \in B\}$).

Schnittmenge mit `intersection`

Die Schnittmenge enthält genau die Elemente, die in beiden Ausgangsmengen gemeinsam vorkommen ($A \cap B = \{x \mid x \in A \wedge x \in B\}$).


```

1 a = {1, 2, 3}
2 b = {3, 4, 5}
3 print(a | b) # {1, 2, 3, 4, 5}

```

```

1 const a = new RecursiveSet<number>(1, 2, 3);
2 const b = new RecursiveSet<number>(3, 4, 5);
3 console.log(a.union(b)); // {1, 2, 3, 4, 5}

```

Listing 48: Vereinigung zweier Mengen in Python und mit RecursiveSet

```

1 print(a & b) # {3}

```

```

1 console.log(a.intersection(b)); // {3}

```

Listing 49: Schnittmenge zweier Mengen in Python und mit RecursiveSet

Differenz mit difference

Die Differenz zweier Mengen enthält alle Elemente der ersten Menge, die nicht in der zweiten Menge vorkommen ($A \setminus B = \{x \mid x \in A \wedge x \notin B\}$).

```

1 print(a - b) # {1, 2}

```

```

1 console.log(a.difference(b)); // {1, 2}

```

Listing 50: Differenz zweier Mengen in Python und mit RecursiveSet

Symmetrische Differenz mit symmetricDifference

Die symmetrische Differenz enthält genau die Elemente, die in einer der beiden Mengen vorkommen, aber nicht in beiden zugleich ($A \triangle B = (A \setminus B) \cup (B \setminus A)$).

Die API in TypeScript macht durch die Methodenbenennung unmittelbar sichtbar, welche mengentheoretische Verknüpfung der Algorithmus ausführt.

3.3.3 Teil- und Obermengenbeziehungen

Eine Menge A heißt Teilmenge von B ($A \subseteq B$), wenn jedes Element aus A auch in B enthalten ist. Umgekehrt heißt A Obermenge von B ($A \supseteq B$), wenn alle Elemente von B in A enthalten sind.

```

1 print(a ^ b) # {1, 2, 4, 5}

```

```

1 console.log(a.symmetricDifference(b)); // {1, 2, 4, 5}

```

Listing 51: Symmetrische Differenz in Python und mit RecursiveSet

In Python lassen sich diese Relationen über die Operatoren `<=` und `>=` ausdrücken. Listing 52 zeigt die entsprechende Gegenüberstellung zur TypeScript-Bibliothek.

```

1 small = {2, 3}
2 large = {1, 2, 3, 4, 5}
3
4 print(small <= large) # True
5 print(large >= small) # True
6 print(large <= small) # False

```

```

1 const small = new RecursiveSet<number>(2, 3);
2 const large = new RecursiveSet<number>(1, 2, 3, 4, 5);
3
4 console.log(small.isSubset(large)); // true
5 console.log(large.isSuperset(small)); // true
6 console.log(large.isSubset(small)); // false

```

Listing 52: Teil- und Obermengenbeziehungen im Sprachvergleich

3.3.4 Auswahl eines beliebigen Elements

In mathematischen Argumentationen wird häufig ein beliebiges Element $x \in A$ betrachtet. Dies darf programmatisch nicht mit dem ersten Element einer Iteration verwechselt werden.

Da `RecursiveSet` intern eine stabile Iterationsreihenfolge besitzt, würde ein Zugriff auf das erste Element stets dasselbe Ergebnis liefern. Für die Zufallsauswahl stellt die Bibliothek daher die Methode `pickRandom()` bereit.

3.3.5 Kartesisches Produkt und Potenzmenge

Das kartesische Produkt ($A \times B$) und die Potenzmenge ($\mathcal{P}(A)$) sind Operationen, die neue Strukturen generieren. Da Python für diese Operationen keine nativen

```

1 import random
2
3 values = {1, 2, 3, 4, 5}
4 arbitrary_element = random.choice(tuple(values))

```

```

1 const values = new RecursiveSet<number>(1, 2, 3, 4, 5);
2 const arbitraryElement = values.pickRandom();

```

Listing 53: Auswahl eines Elements in Python und mit RecursiveSet

Methoden am Set-Typ anbietet, werden sie häufig über Hilfsfunktionen abgebildet. `RecursiveSet` fasst diese Schritte zusammen und stellt beide Operationen als Methoden bereit.

Kartesisches Produkt mit `cartesianProduct`

Das kartesische Produkt zweier Mengen A und B ist die Menge aller geordneten Paare ($A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$).

```

1 main_dishes = {"Burger", "Pasta"}
2 drinks = {"Water", "Cola"}
3
4 meals = {(d, b) for d in main_dishes for b in drinks}

```

```

1 const mainDishes = new RecursiveSet<string>("Burger", "Pasta");
2 const drinks = new RecursiveSet<string>("Water", "Cola");
3
4 const meals = mainDishes.cartesianProduct(drinks);

```

Listing 54: Kartesisches Produkt in Python und mit RecursiveSet

Potenzmenge mit `powerset`

Die Potenzmenge einer Menge A ist die Menge aller Teilmengen von A ($\mathcal{P}(A) = \{X \mid X \subseteq A\}$). Listing 55 zeigt den `powerset()`-Aufruf.

3.3.6 Funktionale Transformationen statt Komprehension

Für die Transformation von Mengen nutzt Python sogenannte Mengenkomprehensionen (Set Comprehensions), um Selektion und Transformation kompakt in einer Zeile zu vereinen. Listing 56 zeigt dieses Grundprinzip: Aus einer Menge von Zahlen sollen alle ungeraden Werte ausgewählt und mit 10 multipliziert werden.

```

1 const toppings = new RecursiveSet<string>("Cheese", "Sauce", "Olives");
2
3 const allPossibleOrders = toppings.powerset();
4 console.log(allPossibleOrders.size); // 2^3 = 8

```

Listing 55: Potenzmenge mit RecursiveSet

```

1 mixed_numbers = {1, 2, 3, 4, 5}
2 odd_tens = {n * 10 for n in mixed_numbers if n % 2 != 0}

```

Listing 56: Python: Mengenkompensation mit Selektion und Transformation

Da sich eine derartige Syntax in TypeScript nicht direkt als Spracherweiterung nachbilden lässt, stellt `RecursiveSet` funktionale Operationen bereit, mit denen sich dieselbe Logik durch Methodenverkettung ausdrücken lässt.

Mathematische Sicht und Arrow Functions

Zur formalen Beschreibung sei A eine Menge, $f: A \rightarrow B$ eine Funktion und $P(x)$ ein Prädikat (eine Aussage über x , die wahr oder falsch ist).

Im TypeScript-Code werden solche Logiken als „Arrow Functions“ (Lambda-Funktionen) formuliert. Die Schreibweise `n => n * 10` liest sich als „Nimm Element n und bilde es ab auf $n \cdot 10$ “. Damit lässt sich die mathematische Mengenschreibweise $\{x \in A \mid P(x)\}$ in Code überführen.

Selektion mit `filter`

Die Methode `filter` wählt aus einer Ausgangsmenge alle Elemente aus, die ein gegebenes Prädikat erfüllen ($\text{filter}(A, P) = \{x \in A \mid P(x)\}$).

```

1 const vocabulary = new RecursiveSet<string>("cat", "dog", "bat", "mouse", "owl");
2
3 const threeLetterWords = vocabulary.filter(word => word.length === 3);

```

Listing 57: Selektion mit `filter`

Abbildung mit `map`

Die Methode `map` wendet eine Funktion auf jedes Element an ($\text{map}(A, f) = \{f(x) \mid x \in A\}$). Da das Ergebnis wiederum eine Menge ist, werden doppelte Resultate zusammengefasst.

```

1 const uppercaseWords = threeLetterWords.map(word => word.toUpperCase());
2
3 // Verschiedene Wörter können dieselbe Länge haben.
4 // Duplikate werden vom Set zusammengefasst:
5 const wordLengths = threeLetterWords.map(word => word.length); // {3}

```

Listing 58: Abbildung mit map

Quantoren mit every und some

Die Bibliothek unterstützt Operationen, die logischen Quantoren entsprechen. Die Methode `every` prüft, ob ein Prädikat für *alle* Elemente gilt ($\forall x \in A : P(x)$), während `some` die Existenz *mindestens* eines Elements testet ($\exists x \in A : P(x)$).

```

1 const numbers = new RecursiveSet<number>(2, 4, 6, 8);
2
3 const areAllEven = numbers.every(n => n % 2 === 0); // true
4 const containsEight = numbers.some(n => n === 8); // true

```

Listing 59: Logische Quantoren mit every und some

Kombination von Selektion und Abbildung mit filterMap

Sind Selektion und Transformation gemeinsam erforderlich, bündelt `filterMap` diese in einem Durchlauf ($\text{filterMap}(A, P, f) = \{f(x) \mid x \in A \wedge P(x)\}$). Dies entspricht der Python-Komprehension aus Listing 56.

```

1 const mixedNumbers = new RecursiveSet<number>(1, 2, 3, 4, 5);
2
3 const oddTens = mixedNumbers.filterMap(
4   n => n % 2 !== 0, // Prädikat
5   n => n * 10       // Transformation
6 ); // {10, 30, 50}

```

Listing 60: Kombination aus Filter und Abbildung mit filterMap

Verschachtelte Konstruktionen mit flatMap

Wenn jedem Eingabewert eine Menge von Ergebnissen zugeordnet wird, fasst `flatMap` diese Teilmengen zu einer Gesamtmenge zusammen ($\text{flatMap}(A, f) = \bigcup_{x \in A} f(x)$). Listing 61 zeigt zunächst die verschachtelte Python-Komprehension.

In TypeScript flacht die Methode `flatMap` die entstehenden Teilmengen direkt in das aufrufende `integerSet` ab.

```
1 integer_set = {0}
2 natural_numbers = [1, 2, 3]
3
4 expanded_set = {val for n in natural_numbers for val in (n, -n)}
5 integer_set |= expanded_set
```

Listing 61: Python: Verschachtelte Mengenkompensation

```
1 const integerSet = new RecursiveSet<number>(0);
2 const naturalNumbers = [1, 2, 3];
3
4 integerSet.flatMap(
5   naturalNumbers,
6   n => new RecursiveSet<number>(n, -n)
7 );
```

Listing 62: Verschachtelte Konstruktion mit flatMap

Aggregation mit reduce

Während die bisher betrachteten Operationen Mengen erzeugen oder logische Aussagen liefern, fasst `reduce` alle Elemente zu einem einzelnen Wert zusammen ($\sum_{x \in A} x$).

```
1 const costs = new RecursiveSet<number>(10, 20, 30);
2
3 const totalSum = costs.reduce((acc, current) => acc + current, 0); // 60
```

Listing 63: Aggregation mit reduce

Die native Kompensationssyntax wird somit durch eine API funktionaler Operationen abgebildet, deren Bedeutung sich direkt auf die mathematische Notation übertragen lässt.

4 Interne Architektur von RecursiveSet

Während Kapitel 3 die fachliche Programmierschnittstelle und die konzeptionelle Notwendigkeit einer wertsemantischen Mengenstruktur behandelte, fokussiert dieses Kapitel die zugrunde liegende technische Architektur. Dabei wird beleuchtet, wie die Bibliothek die Brücke zwischen der formalen mathematischen Theorie und der maschinennahen Ausführung in der JavaScript-Laufzeitumgebung (V8) schlägt.

4.1 Architektonische Grundentscheidungen

Die naheliegendste Frage zu Beginn einer solchen Portierung lautet: Warum wird das native `Set` von TypeScript nicht einfach in eine Wrapper-Klasse gewickelt, um das in Abschnitt 3.1 beschriebene Problem der mangelnden Wertsemantik zu umgehen? Man könnte komplexe Objekte beispielsweise in JSON-Strings umwandeln (`JSON.stringify`) und diese Strings als Repräsentation im nativen Set speichern.

Ein solcher Ansatz scheitert jedoch an zwei fundamentalen Hürden: einer semantischen und einer architektonischen.

Die **semantische Hürde** betrifft das Wesen mathematischer Mengen. Mengen sind fachlich ungeordnet: Die Menge $\{1, 2\}$ ist mathematisch absolut identisch zur Menge $\{2, 1\}$. Eine Serialisierung mittels `JSON.stringify` ist jedoch strikt sequenziell. Sie würde die Repräsentationen `"[1,2]"` und `"[2,1]"` als zwei völlig unterschiedliche Zeichenketten und damit als zwei verschiedene Elemente werten. Um dies zu umgehen, müsste jede geschachtelte Menge vor jedem Lese- oder Schreibzugriff tiefenrekursiv sortiert werden.

Dies führt unweigerlich zur **architektonischen Hürde**: Zwar ist die native `JSON.stringify`-Funktion der JavaScript-Laufzeitumgebung hochgradig optimiert, doch das erzwungene vorherige Sortieren und das ständige Erzeugen neuer String-Objekte bei jedem noch so kleinen Zugriffsobjekt erfordert enorm viel CPU-Zeit. Zudem belastet die ständige Erzeugung temporärer Strings die Speicherverwaltung massiv:

Definition 4.1. Garbage Collection (GC): Ein Prozess der Laufzeitumgebung (wie der V8-Engine), der den Arbeitsspeicher nach Objekten durchsucht, die vom Programm nicht mehr referenziert werden, um deren Speicherplatz wieder freizugeben. Werden in kurzen Abständen extrem viele temporäre Objekte (wie JSON-Strings für Suchabfragen) erzeugt, muss die GC den Programmablauf für Aufräumarbeiten pausieren („Stop-the-World“-Pausen), was zu Latenzen und Einbrüchen in der Performance führt.

Um sowohl die mathematische Gleichheit (ordnungsunabhängige Wertsemantik) als auch eine Zugriffszeit von im Durchschnitt amortisiert $\mathcal{O}(1)$ (unter der Annahme einer guten Hash-Verteilung) zu garantieren, wurde `RecursiveSet` stattdessen als vollständige Eigenimplementierung entwickelt. Nur so erhält die Bibliothek die volle Kontrolle über den Hashing-Algorithmus (der bei Mengen ordnungsunabhängig arbeiten muss) und das Speicherlayout, um unnötige Objekt-Allokationen zu vermeiden.

4.2 Die Hash-Engine (Zentrale Komponente)

Ein zentraler Bestandteil jeder Hashtabelle ist die Hash-Engine. Ihre Aufgabe ist es, aus beliebigen, komplexen Werten deterministische Zahlen (Hash-Werte) zu generieren. Dieser Hash-Wert fungiert als mathematischer Wegweiser, der den Suchraum in der Tabelle massiv eingrenzt, bevor die aufwendigere inhaltliche Gleichheitsprüfung stattfindet.

Die Bibliothek nutzt die zentrale Funktion `getHashCode` als *Dispatcher*. Ein Dispatcher (engl. für Verteiler oder Leitstelle) ist ein Architekturmuster, bei dem eine zentrale Funktion den eingehenden Datentyp untersucht und die Aufgabe an den jeweils spezialisierten, effizientesten Unteralgorithmus weiterleitet.

4.2.1 Ganzzahlen und der ArrayBuffer-Trick für IEEE-754

Wie bereits in Definition 2.1 (Kapitel 2) dargelegt, unterscheidet TypeScript auf Typ-Ebene nicht zwischen Ganzzahlen und Kommazahlen. Alle numerischen Werte werden nach dem IEEE-754-Standard als 64-Bit-Fließkommazahlen behandelt.

Für die Berechnung von Hash-Werten ist dies jedoch eine architektonische Hürde: Klassische bitweise Hash-Operationen (wie das exklusive ODER, `^`) arbeiten in JavaScript ausschließlich mit 32-Bit-Ganzzahlen. Wendet man sie direkt auf eine Fließkommazahl an, schneidet die Laufzeitumgebung die Nachkommastellen schlichtweg ab. Der Hash von `3.1` wäre demnach identisch mit dem Hash von `3.9`, was zu massiven Hash-Kollisionen führen würde.

Die Hash-Engine löst dieses Problem durch zwei strikt getrennte Ausführungspfade. Da dieser Algorithmus, siehe Listing 64, sehr dicht und hardwarenah geschrieben ist, lohnt sich eine schrittweise Betrachtung.

```
1  const _buffer = new ArrayBuffer(8);
2  const _f64 = new Float64Array(_buffer);
3  const _i32 = new Int32Array(_buffer);
4
5  // Mixing constants (derived from MurmurHash3)
6  const HASH_C1 = 0xcc9e2d51;
7  const HASH_C2 = 0x1b873593;
8
9  function getHashCode(val: Value): number {
10     if (typeof val === 'number') {
11         // Fast Path: 32-Bit Ganzzahlen
12         if ((val | 0) === val) {
13             let h = val | 0;
14             h = Math.imul(h ^ (h >> 16), 0x85ebca6b);
15             h = Math.imul(h ^ (h >> 13), 0xc2b2ae35);
16             return (h ^ (h >> 16)) >> 0;
17         }
18
19         // Slow Path: IEEE-754 Fließkommazahlen
20         _f64[0] = val;
21         const low = _i32[0];
22         const high = _i32[1];
23
24         return (Math.imul(low, HASH_C1) ^ Math.imul(high, HASH_C2)) >> 0;
25     }
26     // ...
27 }
```

Listing 64: Hashing von numerischen Werten

Schritt 1: Globale Speichervorbereitung (Zeile 1–7)

Noch bevor die eigentliche Funktion aufgerufen wird, reserviert die Bibliothek mit `new ArrayBuffer(8)` einen 8 Bytes (64 Bits) großen, rohen Block im Arbeitsspeicher. Die typisierten Arrays `_f64` und `_i32` erzeugen dabei keine neuen Daten, sondern fungieren als zwei unterschiedliche „Linsen“ (Sichten) auf exakt denselben Speicherblock. Wofür diese Linsen benötigt werden, zeigt sich im Slow Path. Zusätzlich werden die globalen Konstanten `HASH_C1` und `HASH_C2` definiert, die aus dem bewährten MurmurHash3-Algorithmus entlehnt sind [3].

Schritt 2: Der Fast Path für Ganzzahlen (Zeile 11–17)

Zunächst wird mittels der bitweisen Operation `(val | 0)` geprüft, ob die Zahl trotz des Fließkomma-Formats verlustfrei als 32-Bit-Ganzzahl behandelt werden kann.

Ist dies der Fall, wird der Wert durch sogenanntes *Avalanche-Mixing* (Lawineneffekt) vermischt. Warum ist das nötig? Würde man aufeinanderfolgende Zahlen (wie 1, 2 und 3) unverändert als Hash-Werte nutzen, entstünden in der Hashtabelle schnell sogenannte **Cluster** (Klumpen). Das bedeutet, dass viele Elemente direkt nebeneinander abgelegt werden. Beim späteren Einfügen von Elementen führte dies unweigerlich zu langen Suchen nach Plätzen, da der Algorithmus viele belegte Positionen überspringen muss, bevor er eine Lücke findet.

Um diese Klumpenbildung zu verhindern, nutzt der Code Konstanten wie `0x85ebca6b`. Das Präfix `0x` kennzeichnet dabei lediglich eine **hexadezimale** Zahl, eine kompakte Schreibweise für lange Bitmuster. Diese speziellen Bitmuster sorgen bei der Multiplikation für eine optimale Streuung: Ändert sich an der Eingabezahl nur ein einziges Bit, ändert sich durch den Lawineneffekt fast das gesamte Bitmuster des resultierenden Hash-Werts. So landen auch sehr ähnliche Zahlen auf völlig unterschiedlichen Tabellenplätzen.

Schritt 3: Hardwarenahe Multiplikation mit `Math.imul`

Auffällig ist in Zeile 14 und 15 die ständige Verwendung von `Math.imul`. Ein gewöhnliches Multiplikationszeichen (`*`) würde in JavaScript dazu führen, dass die erzeugten Bitmuster intern wieder als 64-Bit-Fließkommazahlen berechnet werden. Bei großen Multiplikatoren führt das unweigerlich zu Genauigkeitsverlusten (Rundungsfehlern) und macht die unteren Bits des Hash-Werts unbrauchbar. Die Funktion `Math.imul` (Integer Multiplication) zwingt die JavaScript-Engine stattdessen zu einer echten, hardwarenahen 32-Bit-Ganzzahl-Multiplikation. Bits, die durch einen Überlauf über die 32-Bit-Grenze hinausschießen, werden auf CPU-Ebene einfach abgeschnitten, was für Hashing-Algorithmen das gewünschte Verhalten ist.

Schritt 4: Der Slow Path für Fließkommazahlen (Zeile 19–24)

Schlägt die Ganzzahl-Prüfung fehl, greift der Slow Path. Hier kommt nun ein architektonischer Ansatz zum Einsatz, der an ein `union`-Konstrukt aus der Sprache C erinnert. Um die gesamten 64 Bits der Fließkommazahl (Vorzeichen, Exponent und Mantisse) für das Hashing zu nutzen, müssen diese in zwei 32-Bit-Hälften zerlegt werden.

Anstatt die Zahl durch teure mathematische Operationen zu zerspalten, greift die Bibliothek auf den in Schritt 1 vorbereiteten `ArrayBuffer` zurück:

1. Die Fließkommazahl wird durch die 64-Bit-Linse (`_f64[0] = val`) in Zeile 20

nativ in den Speicher geschrieben.

2. Anschließend werden in den Zeilen 21 und 22 exakt dieselben 64 Bits durch die Ganzzahl-Linse (`_i32`) ohne jegliche Konvertierung als zwei fertige 32-Bit-Blöcke (`low` und `high`) wieder ausgelesen.

Dieses direkte Lesen und Schreiben von Speicherbits erzeugt keine temporären Objekte und entlastet den Garbage Collector somit vollständig. Zum Abschluss werden beide Hälften in Zeile 24 über `Math.imul` mit den globalen Konstanten `HASH_C1` und `HASH_C2` überkreuz multipliziert und per exklusivem ODER vereint. Dadurch ist sichergestellt, dass selbst winzigste Änderungen im Exponenten oder in der Mantisse der Fließkommazahl einen neuen Hash-Wert generieren.

4.2.2 Hashing von Strings und strukturellen Objekten

Bei Zeichenketten und komplexen Objekten greifen andere Mechanismen, die ebenfalls auf Performance und Speicherschonung ausgelegt sind, wie in Listing 65 dargestellt.

```
1   if (typeof val === 'string') {
2       let h = 0x811c9dc5;
3       const len = val.length;
4       for (let i = 0; i < len; i++) {
5           h ^= val.charCodeAt(i);
6           h = Math.imul(h, 0x01000193);
7       }
8       return h >>> 0;
9   }
10
11  if (val && typeof val === 'object') {
12      return val.hashCode();
13  }
```

Listing 65: Hashing von Strings und strukturellen Objekten

Für **Strings** verwendet die Bibliothek eine Variante des *FNV-1a*-Algorithmus (Fowler–Noll–Vo). Die Zeichenkette wird in einer Schleife durchlaufen und der numerische Wert jedes Zeichens (`charCodeAt`) wird über ein exklusives ODER (`^=`) in den Hash eingemischt. Der bewusste Verzicht auf Methoden, die den String in Arrays aufspalten oder neue Teilzeichenketten (Sub-Strings) erzeugen, stellt sicher, dass während der Schleife keine temporären Objekte im Arbeitsspeicher reserviert (alloziert) werden müssen. Diese sogenannte *Allokationsfreiheit* entlastet den Garbage Collector.

Bei **komplexen Objekten** (wie verschachtelten Mengen oder Tupeln) delegiert die Hash-Engine die Verantwortung an das Objekt selbst, indem sie schlicht `val.hashCode` abrufen.

Damit der TypeScript-Compiler garantieren kann, dass diese Eigenschaft auf dem übergebenen Objekt überhaupt existiert, stützt sich die Bibliothek auf einen vertraglichen Rahmen: das interne `Structural`-Interface. Dieses ist in Listing 66 dargestellt.

```
1 interface Structural {
2     /**
3      * Ein stabiler Hash-Wert für das Objekt.
4      * Der Zugriff auf diese Eigenschaft friert den Zustand des Objekts
5      * in rekursiven Strukturen in der Regel ein.
6      */
7     readonly hashCode: number;
8
9     /**
10     * Prüft die inhaltliche (strukturelle) Gleichheit mit einem anderen Objekt.
11     */
12     equals(other: unknown): boolean;
13
14     /** * Liefert eine deterministische Textrepräsentation.
15     */
16     toString(): string;
17 }
```

Listing 66: Der Vertrag für komplexe Mengenelemente

Wie in Abschnitt 2.2.3 im Rahmen des statischen Duck-Typings erläutert, fordert TypeScript keine starren Klassenhierarchien. Jedes Objekt, das die in Listing 66 definierten Eigenschaften und Methoden bereitstellt, wird vom Typsystem automatisch als `Structural` anerkannt und kann in der Bibliothek als Schlüssel oder Mengenelement verwendet werden.

Dieser Vertrag definiert exakt die drei Säulen, die eine wertebasierte Hashtabelle benötigt:

1. `hashCode`: Liefert den vorberechneten Hash-Wert zur schnellen Eingrenzung des Suchraums. Das Schlüsselwort `readonly` garantiert dem Typsystem, dass sich dieser Wert nach der Erstellung nicht mehr ändert. Der lesende Zugriff auf diese Eigenschaft ist zudem der Auslöser für den kaskadierenden Freeze-Mechanismus (siehe Abschnitt 4.7).
2. `equals`: Übernimmt die inhaltliche Gleichheitsprüfung, sobald der Hash-Wert eine mögliche Kollision oder einen Treffer anzeigt (siehe Abschnitt 4.4).

3. `toString`: Erzwingt eine konsistente Textrepräsentation, was für die deterministische Konsolenausgabe von tief verschachtelten Strukturen essenziell ist.

Genau dieses Interface ist der Grund, warum sich die in Kapitel 3 vorgestellten Klassen `Tuple` und `RecursiveMap` nahtlos, typsicher und ohne zusätzlichen Konvertierungsaufwand in die Hash-Architektur einfügen lassen.

4.3 Speicherlayout (Structure of Arrays)

Ein klassischer, objektorientierter Entwurf für eine Hashtabelle würde Objekte der Form `{ value, hash }` anlegen. Diesen Ansatz nennt man *Array of Structures (AoS)*. Bei Tausenden von Zuständen in einem Automaten führt dies jedoch dazu, dass der Arbeitsspeicher fragmentiert. Die CPU muss ständig an verschiedene Orte im RAM springen, um die Objekte zu lesen, was zusätzliche Taktzyklen erfordert.

Um die Cache-Lokalität der CPU besser auszunutzen, wird von `RecursiveSet` ein Entwurfsmuster aus dem High-Performance-Computing implementiert:

Definition 4.2. Structure of Arrays (SoA): Bei diesem Architekturmuster werden Objekte nicht als zusammenhängende Datenblöcke im Speicher abgelegt. Stattdessen werden ihre einzelnen Eigenschaften in voneinander getrennten, parallelen Arrays gespeichert. Dies verbessert die Cache-Lokalität, da die CPU beim Durchsuchen einer bestimmten Eigenschaft (z. B. aller Hash-Werte) diese als kompakten Block am Stück aus dem Arbeitsspeicher (RAM) in den schnellen CPU-Cache laden kann [4, S. 161].

```
1 class RecursiveSet<T extends Value> implements Structural, Iterable<T> {
2     // Dichte Arrays (Nutzdaten liegen ohne Lücken nebeneinander)
3     private _values: T[] = [];
4     private _hashes: number[] = [];
5
6     // Indextabelle für die Suchstruktur (Open Addressing)
7     private _indices: Uint32Array;
8
9     private _bucketCount: number;
10    private _mask: number;
11 }
```

Listing 67: Speicherarchitektur im Structure of Arrays (SoA) Format

Um die Effizienz dieser Architektur zu verstehen, müssen die fünf Felder der Klasse getrennt betrachtet werden. Sie teilen sich in zwei logische Ebenen auf:

den eigentlichen Datenbestand (die dichten Arrays) und die Navigationsstruktur (die Indextabelle).

Ebene 1: Der dichte Datenbestand

Die Arrays `_values` und `_hashes` wachsen dicht (ohne Lücken) heran. Wenn die Menge drei Elemente enthält, sind exakt die Indizes 0, 1 und 2 belegt.

- `_values`: Speichert die tatsächlichen, typisierten Elemente der Menge.
- `_hashes`: Speichert die zugehörigen, vorberechneten 32-Bit-Hash-Werte. Das Element an Index 5 in `_values` gehört exakt zum Hash an Index 5 in `_hashes`. Dass die Hashes in einem eigenen Array liegen, beschleunigt die Suche, da die CPU Hash-Werte auf Kollisionen scannen kann, ohne das speicherintensive `_values`-Array laden zu müssen.

Ebene 2: Die Navigationsstruktur

Während die dichten Arrays lediglich die Daten bunkern, kümmert sich die zweite Ebene darum, diese Daten über Hash-Werte effizient wiederzufinden:

- `_indices`: Dies ist die eigentliche Hashtabelle (die Suchstruktur). Sie ist als `UInt32Array` implementiert und funktioniert wie ein Inhaltsverzeichnis. Bemerkenswert ist hierbei, dass sie **keine echten Elemente** speichert, sondern lediglich **Verweise auf Positionen** in den dichten Arrays.

Da ein neu angelegtes `UInt32Array` im Arbeitsspeicher standardmäßig mit Nullen gefüllt ist, nutzt die Bibliothek den Wert 0 als zweckmäßige Markierung für einen „leeren Slot“. Um nun den ersten echten Index (die 0) der dichten Arrays davon unterscheiden zu können, werden alle gespeicherten Positionen intern um den Wert 1 erhöht. Ein Eintrag von 3 in der Indextabelle bedeutet also: „Ein *möglicher* Treffer für diesen Hash liegt in den dichten Arrays an Position 2“. Ob es sich dabei wirklich um das gesuchte Element oder nur um eine Kollision handelt, und wie der Algorithmus weiter navigiert, wird im Abschnitt 4.5 (*Linear Probing*) detailliert beleuchtet.

- `_bucketCount`: Definiert die Gesamtkapazität der `_indices`-Tabelle (die Anzahl der verfügbaren Such-Slots). Diese Kapazität ist zwingend immer eine Zweierpotenz (z. B. 16, 32, 64).
- `_mask`: Die Bitmaske ist ein klassischer Performance-Trick aus der Hardware-Programmierung. Um einen beliebig großen Hash-Wert auf die tatsächliche Größe der Tabelle herunterzubrechen, nutzt man klassischerweise die Modulo-Division (`hash % _bucketCount`). Da Divisionen für die CPU jedoch vergleichsweise teuer sind, wird die Bitmaske auf `_bucketCount - 1` gesetzt (bei

16 Buckets ist die Maske 15, binär 00001111). Die Operation `hash & _mask` (bitweises UND) liefert exakt dasselbe Ergebnis wie Modulo, wird von der CPU als elementare Bit-Operation aber typischerweise in einem einzigen Taktzyklus berechnet.

4.4 Konzepte der Gleichheit und Type Narrowing

Wie in Abschnitt 4.2 beschrieben, dient der Hash-Wert lediglich als Wegweiser, um die Suche in der Hashtabelle einzugrenzen. Da zwei inhaltlich unterschiedliche Mengen rein rechnerisch denselben Hash-Wert generieren können (Kollision), muss die Datenstruktur gefundene Kandidaten iterativ auf Gleichheit prüfen. Hierbei ist eine präzise terminologische Trennung essenziell:

- **Referenzidentität:** Zwei Variablen verweisen auf exakt denselben Bereich im Arbeitsspeicher (`a === b`).
- **Hash-Gleichheit:** Zwei Objekte generieren denselben numerischen Hash-Wert (`a.hashCode === b.hashCode`). Dies ist eine notwendige, aber keine hinreichende Bedingung für weitere Gleichheitsannahmen.
- **Strukturelle Gleichheit:** Zwei getrennte Objekte weisen denselben inhaltlichen Aufbau auf (geprüft über die Methode `equals`).
- **Mathematische Gleichheit:** Die semantische Eigenschaft, dass zwei Mengen dieselben Elemente beinhalten, unabhängig von der Reihenfolge, in der sie im Speicher abgelegt wurden.

Genau diese Aufgabe übernimmt die vertraglich im `Structural`-Interface geforderte `equals`-Methode. Am Beispiel der Klasse `RecursiveSet` zeigt sich, wie TypeScript-spezifische Typsicherheit und Performance-Optimierungen hier Hand in Hand gehen:

Dieser Algorithmus ist wie ein Trichter aufgebaut: Er versucht, eine aufwendige Tiefenprüfung (Zeile 13–15) durch immer strengere, aber effiziente Vorab-Ausschlusskriterien zu vermeiden (*Short-Circuit Evaluation*).

Sicherheit durch `unknown` und Type Narrowing (Zeile 1–6)

Die Signatur der Methode fordert bewusst den Typ `unknown` für den Parameter `other`. Wie in Definition 2.10 (Kapitel 2) erläutert, zwingt dieser Typ den Compiler dazu, jegliche blinde Interaktion mit dem Objekt zu verbieten. Die Methode weiß an diesem Punkt nicht, ob `other` eine Zahl, ein String oder ein komplexes Objekt ist.

Zunächst greift ein schneller Abgleich der **Referenzidentität** (Zeile 3). Zeigen `this` und `other` auf exakt dieselbe Speicheradresse (`===`), ist jede weitere Prüfung obsolet – ein Objekt ist immer gleich zu sich selbst.

```

1      equals(other: unknown): boolean {
2          // 1. Identitäts-Kurzschluss (Referenzidentität)
3          if (this === other) return true;
4
5          // 2. Typprüfung und Type Narrowing
6          if (!(other instanceof RecursiveSet)) return false;
7
8          // 3. O(1) Performance-Schranken (Größe und Hash-Gleichheit)
9          if (this.size !== other.size) return false;
10         if (this.hashCode !== other.hashCode) return false;
11
12         // 4. Tiefe strukturelle und mathematische Prüfung
13         for (const v of this._values) {
14             if (!other.has(v)) return false;
15         }
16
17         return true;
18     }

```

Listing 68: Die equals-Methode von RecursiveSet

Ist dies nicht der Fall, folgt in Zeile 6 der entscheidende Schritt für die Typsicherheit: Die Laufzeitprüfung `other instanceof RecursiveSet`. Ist das Fremdobjekt gar keine Menge, wird sofort `false` zurückgegeben.

Wird dieser Check jedoch passiert, greift das in Definition 2.14 (Kapitel 2) beschriebene **Type Narrowing**. Der TypeScript-Compiler analysiert den Kontrollfluss und weiß ab Zeile 9 mit Sicherheit, dass `other` eine Instanz von `RecursiveSet` ist. Ab hier schaltet der Compiler den typsicheren Zugriff auf Eigenschaften wie `other.size`, `other.hashCode` und die Methode `other.has()` frei.

$\mathcal{O}(1)$ Performance-Schranken (Zeile 8–10)

Bevor die Methode die Iteration über alle Elemente startet, werden zwei Eigenschaften verglichen, die in konstanter Zeit ($\mathcal{O}(1)$) abrufbar sind:

1. **Größe (size):** Mengen mit unterschiedlich vielen Elementen können per Definition nicht gleich sein.
2. **Hash-Gleichheit (hashCode):** Wenn zwei `RecursiveSet`-Instanzen inhaltlich gleich sind, *müssen* sie zwingend denselben Hash-Wert besitzen. Unterscheiden sich die Hashes, kann die Prüfung sofort abgebrochen werden. Da der Hash-Wert der Menge intern gecacht wird (siehe Abschnitt 4.7), erfordert dieser Vergleich nahezu keine Rechenzeit.

Strukturelle und mathematische Prüfung über dichte Arrays (Zeile 12–16)

Erst wenn das Fremdobjekt *sicher* eine Menge gleicher Größe und mit demselben Hash-Wert ist, greift die Methode auf das in Abschnitt 4.3 etablierte SoA-Layout zurück. Die `for`-Schleife iteriert über das dichte `_values`-Array und prüft mittels `other.has(v)`, ob jedes Element der aktuellen Menge auch in der anderen Menge existiert.

Da die Methode `has()` selbst im Durchschnitt amortisiert in $\mathcal{O}(1)$ arbeitet (unter Annahme einer gleichmäßigen Hash-Verteilung), bleibt auch die gesamte **strukturelle und mathematische Gleichheitsprüfung** zweier Mengen im Normalfall sehr effizient und steigt nur im Worst-Case auf einen Aufwand von $\mathcal{O}(N)$.

4.5 Kollisionsauflösung und Index-Verwaltung

Da der Hash-Wert zur Indexberechnung genutzt wird, lässt es sich mathematisch nicht vermeiden, dass zwei unterschiedliche Werte denselben Index in der `_indices`-Tabelle berechnen (das sogenannte Taubenschlagprinzip). Die Datenstruktur muss diese Hash-Kollisionen auflösen.

4.5.1 Open Addressing mit Linear Probing

Um die Cache-Lokalität des SoA-Layouts beizubehalten, wird auf externe Zeigerstrukturen verzichtet und stattdessen eine flache Tabellenverwaltung genutzt:

Definition 4.3. Open Addressing mit Linear Probing (Lineares Sondieren):

Bei diesem Verfahren werden alle Verweise direkt im Array der Hashtabelle gespeichert. Berechnet der Hash einen Index (Slot), der bereits von einem anderen Element belegt ist (eine Kollision), rückt der Algorithmus schrittweise zum nächstmöglichen benachbarten Slot weiter, bis er eine Lücke findet oder den gesuchten Wert entdeckt [5, 293f].

Man kann sich dies wie das Einsortieren von Büchern in einem langen Regal vorstellen: Berechnet das System für ein neues Buch einen Stammplatz, der bereits belegt ist, wird das Buch nicht in einem externen Anbau ausgelagert. Stattdessen prüft man einfach den direkten Nachbarplatz auf der rechten Seite, bis man auf eine Lücke trifft.

Die Methode `has` aus Listing 69 demonstriert diese Suchlogik im Zusammenspiel mit dem zuvor eingeführten SoA-Speicherlayout.

Dieser Suchalgorithmus ist der Programmabschnitt, der zur Laufzeit am häufigsten ausgeführt wird. Da diese Suchlogik die Basis für nahezu jede Interaktion mit der Hashtabelle bildet, sei es beim Hinzufügen (`add`), Entfernen (`remove`) oder

```

1  has(element: T): boolean {
2      const h = getHashCode(element);
3      let idx = h & this._mask; // Start-Slot berechnen
4
5      while (true) {
6          const entry = this._indices[idx];
7          if (entry === 0) return false; // Lücke -> Element fehlt
8
9          const valIndex = entry - 1; // 1-basierten Verweis umrechnen
10
11         // 1. Hash vergleichen (schnell), 2. equals vergleichen (teuer)
12         if (this._hashes[valIndex] === h &&
13             areEqual(this._values[valIndex], element)) {
14             return true; // Treffer
15         }
16
17         // Kollision -> Gehe zum nächsten Slot (mit Wrap-around)
18         idx = (idx + 1) & this._mask;
19     }
20 }

```

Listing 69: Linear Probing am Beispiel der `has`-Methode

beim reinen Nachschlagen (`has`), ist sie auf Effizienz ausgelegt. Sie kommt (mit Ausnahme des Aufrufs von `areEqual` bei einem Kandidaten) ohne weitere Funktionsaufrufe aus. Der Ablauf lässt sich in vier Schritte zerlegen:

Schritt 1: Den Start-Slot berechnen (Zeile 2–3)

Zunächst wird der 32-Bit-Hash des gesuchten Elements berechnet. Da dieser Wert weit außerhalb der Tabellengröße liegen kann, wird er mittels der Bitmaske (`h & this._mask`) auf einen gültigen Start-Index (`idx`) in der `_indices`-Tabelle heruntergebrochen.

Schritt 2: Die Abbruchbedingung der Schleife (Zeile 5–7)

Die `while (true)`-Schleife wirkt potenziell endlos. Hier greift jedoch eine Eigenschaft der Hashtabelle: Die Bibliothek stellt sicher, dass die Tabelle niemals zu mehr als 75 % gefüllt ist (der sogenannte Load-Factor, siehe Abschnitt 4.6). Es existiert also *immer* mindestens ein leerer Slot.

Wird ein Slot mit dem Wert 0 (leer) gefunden, stoppt die Suche sofort und liefert `false`. Warum? Weil das Einsortieren im Regal (*Linear Probing*) garantiert, dass bei einer Kollision alle betroffenen Elemente nahtlos hintereinander abgelegt werden. Eine Lücke bedeutet zwingend, dass der Suchpfad an dieser Stelle endet und das gesuchte Element folglich nicht in der Menge existiert.

Schritt 3: Dereferenzierung und Hash-Vorabgleich (Zeile 9–15)

Ist der Slot belegt (`entry > 0`), wird der 1-basierte Verweis durch `entry - 1` in den korrekten 0-basierten Index (`valIndex`) für die dichten Nutzdaten-Arrays umgerechnet.

Nun zeigt sich ein Vorteil des SoA-Layouts: Anstatt sofort die `areEqual`-Funktion aufzurufen, wird zunächst nur der gespeicherte Hash-Wert (`this._hashes[valIndex]`) mit dem gesuchten Hash (`h`) verglichen. Stimmen die Hashes nicht überein, liegt eine klassische Kollision vor und die Inhaltsprüfung wird übersprungen. Nur wenn die Hashes identisch sind (**Hash-Gleichheit**), wird `areEqual` zur Verifizierung der **strukturellen Gleichheit** aufgerufen.

Schritt 4: Linear Probing und der Wrap-around (Zeile 18)

Stellte sich der belegte Slot als Kollision heraus, muss der Algorithmus den nächsten benachbarten Slot prüfen. Die Anweisung `idx = (idx + 1) & this._mask` inkrementiert den Index nicht nur um 1, sondern sorgt gleichzeitig für den sogenannten **Wrap-around**. Erreicht der Suchlauf das physische Ende des Arrays (das Ende des Regals), setzt das bitweise UND (`& this._mask`) den Wert vollautomatisch auf 0 zurück. Der Algorithmus springt also an den Anfang der Tabelle und setzt die Suche dort fort, ohne dass `if`-Abfragen für Array-Grenzen benötigt werden.

4.5.2 Löschvorgang: Tombstones und Backshift Deletion

Das Entfernen von Elementen aus einer Hashtabelle mit *Open Addressing* ist komplexer als bei verketteten Listen. Nimmt man ein Element einfach aus seinem Slot im Regal (durch Zurücksetzen auf 0), entsteht eine harte Lücke. Dies hat Folgen für das *Linear Probing*: Suchen wir nach einem Element, das aufgrund früherer Kollisionen hinter dieser neuen Lücke einsortiert wurde, bricht die `while`-Schleife an der leeren Stelle ab und meldet, das Element existiere nicht. Die Sondierungskette ist gerissen.

Der klassische Lösungsansatz für dieses Problem ist die Verwendung sogenannter Tombstones:

Definition 4.4. Tombstones (Grabsteine): Anstatt einen Slot im Regal wirklich zu leeren (0), wird ein spezielles Marker-Objekt (der Tombstone) an dieser Stelle platziert. Dieser Marker signalisiert dem Suchalgorithmus: „Hier wurde zwar etwas gelöscht, aber brich die Suche nicht ab, sondern suche im nächsten Slot weiter.“

Obwohl Tombstones einfach zu implementieren sind, führen sie bei vielen Einfüge- und Löschvorgängen dazu, dass die Tabelle über die Zeit mit Grabstei-

nen „verschmutzt“. Die Suchwege werden künstlich in die Länge gezogen, was die Zugriffszeiten verschlechtert. Zudem müssen Tombstones regelmäßig durch vollständige Neu-Aufbauten der Tabelle (Rehashing) entfernt werden.

Die Lösung: Backshift Deletion

Um dies zu vermeiden, nutzt die Bibliothek die *Backshift Deletion* (Rückwärts-Verschiebung) – eine Technik, die auf dem *Robin Hood Hashing* aufbaut [6].

Wird ein Element gelöscht, entsteht zunächst eine Lücke. Der Algorithmus, siehe Listing 70, blickt nun in die nachfolgenden Slots und prüft, ob Elemente weiter hinten im Regal stehen, die eigentlich weiter vorne (also näher an der Lücke oder genau darin) stehen müssten. Ist das der Fall, rücken diese Elemente auf, und die Lücke wandert weiter nach hinten, bis sie das Ende der Kollisionskette erreicht und geleert werden kann.

```
1 private removeIndex(holeIdx: number): void {
2     let i = (holeIdx + 1) & this._mask;
3
4     while (this._indices[i] !== 0) {
5         const entry = this._indices[i];
6         const valPtr = entry - 1;
7         const h = this._hashes[valPtr];
8
9         const idealIdx = h & this._mask;
10
11         const distToHole = (holeIdx - idealIdx + this._bucketCount) & this._mask;
12         const distToI = (i - idealIdx + this._bucketCount) & this._mask;
13
14         if (distToHole < distToI) {
15             this._indices[holeIdx] = entry;
16             holeIdx = i;
17         }
18         i = (i + 1) & this._mask;
19     }
20     this._indices[holeIdx] = 0;
21 }
```

Listing 70: Reparatur der Sondierungskette (Backshifting)

Dieser Algorithmus ist ein typisches Beispiel für hardwarenahe Optimierung in TypeScript. Da er ohne Funktionsaufrufe auskommt, übernimmt die Bit-Mathematik die Steuerung. Der Ablauf lässt sich in fünf Phasen unterteilen:

1. **Start der Sondierung (Zeile 2):** Der Algorithmus startet nicht bei der Lücke (`holeIdx`), sondern prüft direkt den ersten rechten Nachbarn. Das bitweise

UND (`& this._mask`) sichert auch hier den zirkulären Wrap-around ab, falls die Lücke am äußersten Ende des Arrays lag.

2. **Abbruchbedingung und Dereferenzierung (Zeile 4–7):** Die Schleife iteriert, bis sie auf einen nativ leeren Slot (0) trifft. Ist ein Slot belegt, wird der 1-basierte Verweis (`entry`) wieder in den 0-basierten Index (`valPtr`) für das dichte `_hashes`-Array umgerechnet, um den Hash-Wert des aktuellen Elements auszulesen.
3. **Ideal-Slot und zirkuläre Distanz-Metrik (Zeile 9–13):** In Zeile 9 wird berechnet, an welchem Index das Element ursprünglich hätte landen sollen (`idealIdx`). Danach folgt der mathematische Kern: Wie weit ist das Element aktuell von seinem Ideal-Slot entfernt (`distToI`) und wie weit wäre es entfernt, wenn es in die Lücke rutschen würde (`distToHole`)?

Die Addition von `this._bucketCount` ist hier essenziell: Da das Array zirkulär ist, könnte `holeIdx - idealIdx` negativ werden. In JavaScript würde der Modulo-Operator bei negativen Zahlen ungeeignete Ergebnisse liefern. Die Addition stellt sicher, dass der Wert positiv bleibt, bevor die Bitmaske ihn wieder in den zirkulären Ring umrechnet.
4. **Die Robin-Hood-Bedingung (Zeile 15–18):** Nun greift die eigentliche *Backshift*-Logik. Ist die Distanz zur Lücke kleiner als die aktuelle Distanz, also gilt `distToHole < distToI`, profitiert das Element von einem Umzug. Es wird physisch in die Lücke verschoben (Zeile 16). Die Lücke wandert dadurch auf die alte Position des Elements (`holeIdx = i`) und wird im nächsten Schleifendurchlauf weiter nach hinten gereicht.
5. **Abschluss (Zeile 21):** Sobald die Schleife auf einen nativ leeren Slot trifft, ist die Kette repariert. Die nach hinten durchgereichte Lücke befindet sich nun am Ende der Sondierungskette und kann mit 0 (leer) überschrieben werden.

Nutzdaten entfernen: Der Dense-Swap

Nachdem die `removeIndex`-Methode die Sondierungskette repariert hat, muss das physische Element aus den dichten Nutzdaten-Arrays (`_values` und `_hashes`) gelöscht werden, ohne Lücken zu hinterlassen.

Ein nativer Aufruf wie `this._values.splice()` würde das Element zwar entfernen, müsste danach aber alle nachfolgenden Elemente im Arbeitsspeicher um einen Platz nach vorne rücken. Dies würde zu einer linearen Laufzeit von $\mathcal{O}(N)$ führen.

Stattdessen nutzt die Bibliothek den **Dense-Swap** (Tausch im dichten Array). Das zu löschende Element wird mit dem *allerletzten* Element des Arrays überschrieben. Danach wird das Array mittels `pop()` in konstanter Zeit ($\mathcal{O}(1)$) am Ende

um einen Platz verkürzt. Da sich durch diesen Tausch die physische Position des letzten Elements geändert hat, muss abschließend nur noch der zugehörige Verweis in der Indextabelle auf den neuen Index aktualisiert werden.

Durch diese Kombination aus *Backshift Deletion* und *Dense-Swap* garantiert die Bibliothek eine stabile Performance bei Löschvorgängen.

4.6 Dynamische Kapazitätsverwaltung (Resizing und Rehashing)

Damit das *Linear Probing* (Abschnitt 4.5.1) performant bleibt und die `while`-Schleife nicht endlos läuft, muss ein gewisser Anteil der Indextabelle leer bleiben. Je voller die Tabelle wird, desto wahrscheinlicher werden Kollisionen und desto länger werden die Sondierungsketten.

Gesteuert wird dies über den sogenannten *Load Factor*:

Definition 4.5. Load Factor (Füllgrad): Der Load Factor beschreibt das Verhältnis von gespeicherten Elementen zur Gesamtkapazität der Hashtabelle ($N/\text{Capacity}$). Die Bibliothek definiert eine feste Obergrenze von 0.75 (75 %). Sobald ein neues Element diesen Schwellenwert überschreitet, verdoppelt die Tabelle ihre Kapazität.

Gleichzeitig arbeitet die Bibliothek speicherschonend: Fällt der Füllgrad durch Löschvorgänge unter 0.25 (25 %), wird die Tabellengröße halbiert (*Shrinking*), sofern eine Mindestgröße (z. B. 16 Slots) nicht unterschritten wird. Da die Kapazität immer verdoppelt oder halbiert wird, bleibt sie stets eine strikte Zweierpotenz, was die Voraussetzung für den Einsatz der Bitmaske (`_mask`) ist.

Das Rehashing: Ein Umzug ohne Inhaltskontrolle

Wenn die Tabelle wächst oder schrumpft, können die alten Indizes nicht direkt kopiert werden. Da sich die Kapazität (`_bucketCount`) ändert, ändert sich unweigerlich auch die Bitmaske (`_mask`). Ein Hash-Wert, der zuvor auf Slot 3 abgebildet wurde, könnte in der neuen Tabelle auf Slot 19 verweisen.

Die Suchstruktur muss daher neu berechnet werden (Rehashing):

Dieser Algorithmus verdeutlicht einen architektonischen Vorteil des *Structure of Arrays*-Layouts (SoA). Da die Nutzdaten (`_values` und `_hashes`) als dicht gepackte Arrays vorliegen, müssen sie beim Rehashing nicht verschoben werden. Der Umzug betrifft ausschließlich die flache `_indices`-Tabelle.

Zudem profitiert der Code von einem signifikanten Performance-Vorteil: Da das `_values`-Array per Definition bereits bereinigt ist und ausschließlich eindeutige Elemente enthält, kann der Algorithmus die `areEqual`-Aufrufkette vollständig igno-

```

1  private resize(newCapacity: number): void {
2      this._bucketCount = newCapacity;
3      this._mask = newCapacity - 1;
4
5      // Frische Indextabelle anlegen (standardmäßig mit Nullen gefüllt)
6      this._indices = new Uint32Array(newCapacity);
7
8      // Alle existierenden Elemente neu einsortieren
9      for (let i = 0; i < this._values.length; i++) {
10         const h = this._hashes[i];
11         let idx = h & this._mask; // Neuen Ideal-Slot berechnen
12
13         // Vereinfachtes Linear Probing ohne equals-Prüfung
14         while (this._indices[idx] !== 0) {
15             idx = (idx + 1) & this._mask;
16         }
17
18         this._indices[idx] = i + 1; // 1-basierten Verweis speichern
19     }
20 }

```

Listing 71: Der Rehashing-Prozess bei einer Kapazitätsänderung

rieren. Er prüft bei einer Kollision lediglich, ob der Slot leer ist (`!== 0`), und rückt weiter.

Amortisierte Laufzeit

Auf den ersten Blick verletzt das Rehashing das Versprechen einer Hashtabelle, Elemente im Durchschnitt in $\mathcal{O}(1)$ einzufügen. Muss die Tabelle wachsen, erfordert die `for`-Schleife einen Aufwand von $\mathcal{O}(N)$, da alle N Elemente neu einsortiert werden.

In der Informatik wird dieses Verhalten über die *amortisierte Laufzeitanalyse* bewertet:

Definition 4.6. Amortisierte Laufzeit: Dieser Begriff beschreibt den durchschnittlichen Aufwand einer Operation über eine sehr lange Sequenz von Aufrufen. Da die Tabelle ihre Größe jedes Mal verdoppelt, passieren Resizing-Operationen seltener. Die vereinzelt $\mathcal{O}(N)$ -Aufwände „amortisieren“ (verteilen) sich auf die weitaus zahlreicheren, sehr effizienten $\mathcal{O}(1)$ -Einfügungen. Im Durchschnitt bleibt der Aufwand für eine einzelne Einfügung somit bei amortisiert $\mathcal{O}(1)$.

Die Kombination aus Verdoppeln beim Wachsen, Halbieren beim Schrumpfen

und Verzicht auf Inhaltsprüfungen während des Rehashings garantiert architektonisch ein Gleichgewicht zwischen Speicherhaushalt und Zugriffsgeschwindigkeit.

4.7 Technische Umsetzung des Freeze-Contracts

In Abschnitt 3.3.1 wurde der *Freeze-Contract* aus API-Sicht eingeführt. Dieser besagt, dass Mengen aus Sicherheitsgründen unveränderlich werden, sobald sie in eine andere Struktur geschachtelt werden. Technisch wird dieser Mechanismus über die Abfrage des Hash-Werts gesteuert.

Da `RecursiveSet` eine ungeordnete Menge repräsentiert, darf der Hash-Wert nicht von der historischen Einfügereihenfolge abhängen. Dies wird durch eine fortlaufende XOR-Verknüpfung (`_xorHash ^= h;`) beim Einfügen sichergestellt, da die XOR-Operation kommutativ ist.

Der tatsächliche Freeze-Vorgang wird ausgelöst, sobald eine übergeordnete Struktur den Getter `get hashCode()` der Kind-Menge aufruft:

```
1  get hashCode(): number {
2      if (this._isFrozen) return this._xorHash;
3
4      // Iteriert über alle Elemente und berührt deren hashCode,
5      // wodurch der Freeze-Effekt nach unten durchgereicht wird.
6      for (let i = 0; i < this._values.length; i++) {
7          const v = this._values[i];
8          if (v && typeof v === 'object') {
9              v.hashCode;
10         }
11     }
12
13     this._isFrozen = true;
14     return this._xorHash;
15 }
```

Listing 72: Einfrieren durch Hash-Abruf

Sobald `this._isFrozen` auf `true` gesetzt ist, blockieren die Methoden `add` und `remove` mit einer Fehlermeldung.

Die `for`-Schleife ruft proaktiv `v.hashCode` auf allen gespeicherten Objekten auf. Dies erzeugt einen kaskadierenden Effekt:

Definition 4.7. Kaskadierung (Cascading): Abgeleitet vom Begriff der Kaskade (einem stufenförmigen Wasserfall) beschreibt dies in der Informatik einen Effekt, der sich automatisch von einer oberen Ebene über alle darunterliegenden Ebenen fortpflanzt. Wenn die Eltern-Menge eingefroren wird, friert sie automatisch alle in ihr enthaltenen Kind-Strukturen ein. Diese frieren wiederum ihre eigenen Kinder ein, bis das Netz der verschachtelten Datenstrukturen versiegelt ist.

Da der `hashCode`-Getter bei allen strukturellen Elementen wie `RecursiveSet`, `Tuple` oder `RecursiveMap` exakt denselben Mechanismus beinhaltet, kaskadiert das Einfrieren durch den gesamten Baum hinab.

5 Vom Python-Werkzeug *PLY* zu Lezer

Nachdem in den vorangegangenen Kapiteln die zentralen Datenstrukturen zur wertsemantischen Modellierung von Mengen und Automaten eingeführt wurden, richtet sich der Blick nun auf die syntaktische Verarbeitung von Eingabetexten.

In den ursprünglichen Python-Implementierungen der Lehrveranstaltung werden wesentliche Schritte der Sprachverarbeitung mit der Bibliothek *PLY* (Python Lex-Yacc) umgesetzt. *PLY* ist eine Parsergenerator-Bibliothek für Python, die sich an den klassischen Werkzeugen *Lex* und *Yacc* orientiert. Diese Werkzeuge werden in Lehrkontexten häufig eingesetzt, um grundlegende Konzepte der Compilertechnik wie Tokenisierung und Parsing praktisch zu vermitteln. Die Bibliothek wurde von David Beazley entwickelt und ist frei verfügbar [7].

Die Architektur eines solchen Sprachprozessors besteht im Kern aus zwei aufeinanderfolgenden Phasen: der lexikalischen Analyse und der darauf aufbauenden syntaktischen Analyse.

Definition 5.1. Lexikalische und syntaktische Analyse: Die lexikalische Analyse (durchgeführt vom *Lexer*) ist der erste Verarbeitungsschritt. Sie zerlegt den Quelltext in eine strukturierte Folge von *Token*, also in kleinste bedeutungstragende Einheiten wie Schlüsselwörter, Zahlen oder Operatoren. Die syntaktische Analyse (durchgeführt vom *Parser*) verarbeitet diese Tokenfolge weiter und überprüft anhand einer formalen Grammatik, ob sie einen gültigen Ausdruck der Sprache bildet. Dabei entsteht eine baumartige Repräsentation der syntaktischen Struktur [8, 5f].

PLY bildet diese beiden Komponenten in seiner Architektur explizit ab:

- **Lexikalische Analyse (Lexer):** Token werden durch reguläre Ausdrücke beschrieben. Der Lexer erkennt auf dieser Grundlage elementare Spracheinheiten wie Zahlen, Operatoren oder Schlüsselwörter.
- **Syntaxanalyse (Parser):** Die Grammatik der Sprache wird als Menge von Produktionsregeln formuliert. *PLY* generiert daraus einen Parser, der auf LR-Parsing-Techniken basiert. Diese Verfahren lesen die Eingabe von links nach rechts und konstruieren eine rechtsableitende Analyse, wodurch Programmiersprachen deterministisch verarbeitet werden können.

Ein charakteristisches Merkmal von *PLY* besteht darin, dass Token-Definitionen und Grammatikregeln unmittelbar im ausführbaren Python-Code formuliert werden (siehe Listing 73). Produktionsregeln erscheinen dabei als gewöhnliche Python-

Funktionen. Die eigentliche Grammatikregel wird dem Parsergenerator über den zugehörigen *Docstring* übermittelt.

Definition 5.2. Docstring (Documentation String): In Python ist ein Docstring eine Zeichenkette, die direkt nach der Definition einer Funktion oder Klasse steht und in erster Linie der Dokumentation dient. *PLY* nutzt diesen Mechanismus, indem die Bibliothek diese Zeichenketten zur Laufzeit ausliest und als Grammatikregeln interpretiert.

Dieser Ansatz erspart zwar separate Grammatikdateien, ist im Lehrkontext jedoch nicht immer ideal. In der Theoretischen Informatik werden kontextfreie Grammatiken üblicherweise in einer kompakten formalen Notation wie der *Erweiterten Backus-Naur-Form* (EBNF) beschrieben. *PLY* verteilt diese Regeln dagegen über einzelne Python-Funktionen und bindet den eigentlichen Regeltext an deren Docstrings. Dadurch tritt die formale Struktur der Grammatik im Quelltext weniger geschlossen hervor.

```
1 tokens = ('NUMBER', 'PLUS')
2
3 t_PLUS = r'\+'
4
5 def t_NUMBER(t):
6     r'\d+'
7     t.value = int(t.value)
8     return t
```

Listing 73: Ply-Grammatik in Python

Während *PLY* somit eine klassische Compilerarchitektur innerhalb der Python-Umgebung realisiert, verfolgt das TypeScript-Ökosystem einen anderen Ansatz. Für editornähe Anwendungen und webbasierte Entwicklungsumgebungen wird häufig das Parser-Framework *Lezer* eingesetzt.

Lezer verfolgt das Ziel, syntaktische Analysen effizient und inkrementell durchzuführen. Im Unterschied zu traditionellen Parsergeneratoren wird der Parser so konstruiert, dass Änderungen am Quelltext lokal verarbeitet werden können, ohne den gesamten Syntaxbaum neu berechnen zu müssen [9].

Ein wesentlicher Vorzug dieses Frameworks liegt in der Art der Sprachspezifikation. Die Grammatik wird nicht mit dem Host-Code vermischt, sondern als deklarative Grammatikbeschreibung in einer eigenen Syntax formuliert. Diese Syntax orientiert sich eng an der theoretischen Darstellung formaler Grammatiken.

Definition 5.3. Erweiterte Backus-Naur-Form (EBNF): Die EBNF ist eine formale Metasprache zur Beschreibung kontextfreier Grammatiken. Sie erweitert die klassische BNF um kompakte Notationsmittel für Wiederholungen, Alternativen, optionale Elemente und Gruppierungen. Dadurch lassen sich Syntaxregeln präziser und übersichtlicher formulieren.

Wie Listing 74 zeigt, ermöglicht *Lezer* eine Formulierung der Grammatikregeln, die – bei geeigneter Strukturierung – in weiten Teilen an die EBNF-nahe Darstellung aus der theoretischen Informatik anschließt:

```
1 @top Expression { Add }
2
3 Add {
4     Expression "+" Expression
5 }
```

Listing 74: Lezer-Grammatik in TypeScript

Aus dieser deklarativen Beschreibung generiert *Lezer* anschließend einen ausführbaren Parser für das TypeScript-Ökosystem.

Beide Ansätze, *PLY* und *Lezer*, basieren formal auf kontextfreien Grammatiken und klassischen Parsingverfahren. Sie unterscheiden sich jedoch deutlich in ihrer Repräsentation und in ihrer Zielumgebung. Während *PLY* vor allem für Python-basierte Compilerprototypen entwickelt wurde und die Grammatik in ausführbaren Code einbettet, ist *Lezer* auf die deklarative und inkrementelle Analyse von Programmtexten ausgerichtet.

Im Rahmen dieser Arbeit dient *PLY* daher als Referenzimplementierung der ursprünglichen Python-Notebooks, während *Lezer* die Grundlage für die Umsetzung der Sprachverarbeitungsschritte im TypeScript-Ökosystem bildet.

5.1 Grundlagen von Lezer

Lezer ist ein Parser-Framework aus dem Umfeld des CodeMirror-Projekts, das auf den Einsatz in modernen Entwicklungswerkzeugen ausgerichtet ist [9]. Im Unterschied zu vielen klassischen Parsergeneratoren steht dabei nicht die einmalige Analyse eines abgeschlossenen Programms im Vordergrund, sondern die Verarbeitung sich fortlaufend verändernder Programmtexte.

Ein zentrales Ziel von *Lezer* besteht darin, die syntaktische Struktur eines Textes bereits während seiner Bearbeitung im Editor verfügbar zu machen. Funktionen wie Syntax-Highlighting, automatische Einrückung oder Fehlermarkierungen setzen eine konsistente Analyse des aktuellen Dokumentzustands voraus. Da sich

der Quelltext während der Bearbeitung fortlaufend verändert, muss der Parser diese Änderungen ressourcenschonend verarbeiten können.

Inkrementelles Parsing

Eine prägende Eigenschaft von *Lezer* ist das sogenannte *inkrementelle Parsing*.

Definition 5.4. Inkrementelles Parsing: Ein Analyseverfahren, bei dem nach einer lokalen Änderung am Quelltext nicht das gesamte Dokument erneut geparkt wird. Stattdessen identifiziert der Parser die von der Änderung betroffenen Bereiche und aktualisiert gezielt nur jene Teilbäume des bestehenden Syntaxbaums, die dadurch ungültig geworden sind.

Dieser Ansatz reduziert den Rechenaufwand und ermöglicht eine fortlaufende Aktualisierung der Syntaxstruktur. Gerade bei größeren Dokumenten oder komplexeren Grammatiken ergibt sich daraus ein spürbarer Performance-Vorteil gegenüber Parserarchitekturen, die bei jeder Änderung den gesamten Text erneut analysieren [10].

Baumrepräsentation der Syntax

Das Ergebnis der Analyse durch *Lezer* ist eine baumartige Repräsentation der Struktur des Eingabetextes. Für die spätere Weiterverarbeitung ist dabei eine präzise terminologische Unterscheidung erforderlich:

Definition 5.5. Concrete Syntax Tree (CST) vs. Abstract Syntax Tree (AST): Ein CST (konkreter Syntaxbaum) repräsentiert den vollständigen Ableitungsbaum der Grammatik einschließlich aller Zwischensymbole, Klammern und sonstigen syntaktischen Hilfszeichen. Ein AST (abstrakter Syntaxbaum) ist demgegenüber eine verdichtete Darstellung, die nur jene Informationen enthält, die für die Programmlogik wesentlich sind, etwa Operatoren und Operanden.

Lezer erzeugt zunächst stets einen konkreten Syntaxbaum (CST). Diese vollständige Repräsentation ist insbesondere für editornahe Funktionen relevant, da sie eine präzise Lokalisierung einzelner Sprachelemente im Quelltext erlaubt. Für die spätere Interpretation oder Auswertung wird dieser CST in der Regel in einen kompakteren AST überführt.

Integration in das JavaScript- und TypeScript-Ökosystem

Ein weiterer Aspekt von *Lezer* ist seine enge Einbindung in das JavaScript- und TypeScript-Ökosystem. Der generierte Parser kann unmittelbar in Webanwendungen oder Entwicklungswerkzeuge wie Jupyter-Notebooks integriert werden.

In der vorliegenden Portierung wird *Lezer* verwendet, um die syntaktische

Analyse der ursprünglichen Lehrbeispiele in eine TypeScript-basierte Implementierung zu übertragen. Die formale Grammatik dient dabei als Ausgangspunkt für die Parsergenerierung, während TypeScript die Weiterverarbeitung der analysierten Strukturen übernimmt.

5.2 Grammatikdefinition in Lezer

Die syntaktische Struktur der zu analysierenden Sprache wird in *Lezer* durch eine deklarative Grammatikbeschreibung festgelegt. Diese Beschreibung bildet die Grundlage für die spätere Generierung des Parsers.

In der vorliegenden Umsetzung wurde die Grammatik direkt innerhalb der verwendeten Jupyter-Notebooks entwickelt. Dieses Vorgehen orientiert sich an den ursprünglichen Lehrmaterialien, die ebenfalls notebookbasiert aufgebaut sind. Ein wesentliches Merkmal der Implementierung besteht darin, dass die Grammatik nicht in einer separaten Datei abgelegt, sondern als mehrzeiliger String direkt im TypeScript-Code definiert wird. Sie liegt damit als Datenobjekt innerhalb des Programms vor und kann ohne weiteren Zwischenschritt an die Parsergenerierung übergeben werden.

Struktur einer Lezer-Grammatik

Eine *Lezer*-Grammatik besteht aus mehreren Komponenten, die gemeinsam die Syntax der Sprache beschreiben:

- der Definition eines Startsymbols,
- Regeln zur Beschreibung komplexerer syntaktischer Strukturen,
- sowie Token-Definitionen für elementare Sprachelemente.

Listing 75 zeigt eine vereinfachte Grammatikdefinition für arithmetische Ausdrücke, wie sie im Notebook als String festgehalten wird:

Das Schlüsselwort `@top` definiert das Startsymbol der Grammatik und damit den Einstiegspunkt der syntaktischen Analyse. Die Regel `Add` beschreibt eine mögliche Struktur eines Ausdrucks. Die Token werden im Block `@tokens` über reguläre Ausdrücke definiert, die die elementaren Spracheinheiten der Eingabe festlegen.

Didaktische Strukturierung durch Zeichenketten-Konkatenation

Während *PLY* Grammatikregeln durch die Verteilung auf einzelne Python-Funktionen ohnehin in kleinere Einheiten aufteilt, erwartet *Lezer* bei der Initialisierung eine zusammenhängende Grammatikbeschreibung als Zeichenkette.

```

1  const grammarDefinition = `
2      @top Expression { Add }
3
4      Add {
5          Expression "+" Expression |
6          Number
7      }
8
9      @tokens {
10         Number { "[0-9]+" }
11     }
12 `;

```

Listing 75: Umsetzung Lezer-Grammatik in TypeScript

Um in den Lehr-Notebooks dennoch eine schrittweise Einführung der Sprachkonstrukte zu ermöglichen, wird die deklarative Grammatik in logische Teilblöcke gegliedert, etwa in Token-Definitionen, Ausdrucksregeln und Anweisungen. Diese Teilstrings werden in aufeinanderfolgenden Notebook-Zellen definiert, jeweils erläutert und schließlich durch Zeichenketten-Konkatenation zur vollständigen Grammatik zusammengesetzt, bevor der Parser generiert wird.

Dieser Ansatz verbindet die formale Geschlossenheit einer EBNF-nahen Notation mit den didaktischen Anforderungen einer schrittweisen Wissensvermittlung im Notebook-Format.

5.3 Generierung des Parsers

Nachdem die Grammatik definiert wurde, kann daraus ein ausführbarer Parser erzeugt werden. In klassischen Projekten geschieht dies häufig über ein separates Generatorwerkzeug im Build-Prozess. In der vorliegenden Umsetzung erfolgt die Parsergenerierung jedoch direkt innerhalb der Notebook-Umgebung.

Hierfür ist insbesondere das Paket `@lezer/generator` relevant. Nach der offiziellen Dokumentation übernimmt dieses Paket die Erzeugung der Parse-Tabellen aus einer Grammatikbeschreibung und kann dazu unmittelbar als JavaScript-Funktion verwendet werden [9].

Die Erstellung des Parsers erfolgt über die Funktion `buildParser`, die die zuvor definierte Grammatik als Eingabe erhält (siehe Listing 76):

Auf dieser Grundlage erzeugt *Lezer* die internen Parsingtabellen und Datenstrukturen. Der resultierende Parser basiert zur Laufzeit auf dem *Lezer-LR*-System, während die erzeugten Syntaxbäume auf den gemeinsamen Baumstrukturen des Ökosystems beruhen [9].

```
1 import { buildParser } from "@lezer/generator";  
2  
3 const parser = buildParser(grammarDefinition);
```

Listing 76: Erstellung des Parsers in TypeScript

Das Ergebnis ist ein Parserobjekt, das Eingabetexte verarbeitet und in eine Baumstruktur überführt. Die Generierung innerhalb des Notebooks bietet den Vorteil, dass Änderungen an der Grammatik unmittelbar getestet werden können. Dieses Vorgehen entspricht der didaktischen Struktur der Lehrmaterialien, in denen Sprachkonstrukte schrittweise eingeführt und direkt erprobt werden.

5.4 Integration in die TypeScript-Implementierung

Die Generierung des Parsers stellt im Projekt keinen isolierten Endpunkt dar, sondern ist in die weitere TypeScript-Implementierung eingebettet. Der erzeugte Parser bildet die Eingangsstufe für die Verarbeitung der Sprachelemente. Dabei greifen mehrere Pakete des *Lezer*-Ökosystems zusammen: `@lezer/generator` erzeugt den Parser, `@lezer/common` stellt mit dem `TreeCursor` eine Schnittstelle zur Traversierung der Baumstruktur bereit, und `@lezer/lr` wird zur expliziten Typisierung der Parserinstanz verwendet [9].

Parsing von Eingabetexten

Nach der Erzeugung der Parserinstanz kann ein Eingabetext analysiert werden. Der Prozess beginnt mit dem Aufruf der Methode `parse`, die einen CST liefert. Um diesen für die Programmlogik nutzbar zu machen, wird er anschließend in einen AST transformiert. Listing 77 zeigt die Kapselung dieses Vorgangs einschließlich der erforderlichen Konfiguration.

In diesem Aufbau übernimmt die Funktion `parse` die Rolle einer Fassade, welche die Komplexität der Baumtransformation kapselt. Die zugrunde liegende Funktion `cst2ast` stützt sich dabei auf drei compilerbauliche Konzepte, die über ihre Parameter gesteuert werden:

Definition 5.6. Terminal-Extraktion (`input`): Lezer speichert in den Blattknoten des CST aus Effizienzgründen nicht unmittelbar die zugehörigen Zeichenketten, sondern lediglich Start- und Endpositionen im Eingabetext. Um die tatsächlichen Werte, etwa Bezeichner oder Zahlenlitterale, für den AST zu rekonstruieren, muss daher der ursprüngliche Quelltext übergeben und an den entsprechenden Positionen ausgelesen werden.


```

1 import { buildParser } from "@lezer/generator";
2
3 // 1. Konfiguration der Transformationsregeln
4 const myOperators = ["IF", "WHILE", ":", "+", "-", "*", "/", "%", "==", "!="];
5 const myListVars = ["Program", "NeExprList", "ExprList"];
6
7 function parse(input: string): AST {
8     // 2. Erzeugung des Concrete Syntax Tree (CST) durch Lezer
9     const tree = parser.parse(input);
10
11     // 3. Rekursive Transformation in das Ziel-Format (AST-Tupel)
12     return cst2ast(tree.cursor(), input, myOperators, myListVars);
13 }

```

Listing 77: Vollständige Integrationslogik in TypeScript

Definition 5.7. Infix Operator Promotion (myOperators): Ein Verfahren der AST-Konstruktion, bei dem ein im CST flach dargestellter Infix-Operator zur Wurzel des resultierenden abstrakten Teilbaums erhoben wird. Erkennt `cst2ast` einen Eintrag aus der Liste `myOperators`, wird ein Ausdruck der Form **A “+” B** in die abstrakte Struktur `["+ ", A, B]` überführt.

Definition 5.8. Listen-Faltung (myListVars): Die Umwandlung einer flachen Folge von Elementen in eine rekursiv verschachtelte Listenstruktur. Diese Darstellungsform orientiert sich an klassischen Konzepten der funktionalen Programmierung (sogenannten *Cons-Zellen*). Anstatt Elemente in einem flachen Array (z. B. `[A, B, C]`) zu belassen, werden sie systematisch in verschachtelte Tupel der Form `["·", Kopf, Rest]` überführt (z. B. `["·", A, ["·", B, ...]]`). Der *Kopf* (Head) enthält das aktuelle Element, der *Rest* (Tail) den rekursiven Verweis auf die restliche Liste. Bezeichner aus `myListVars` lösen diese Faltung aus, da sie die Rechtsrekursion formaler Grammatiken natürlich abbildet und die spätere rekursive Auswertung im Interpreter stark vereinfacht.

Transformation in das projektspezifische AST-Format

Das Ziel dieser Mechanismen besteht darin, den abstrakten Syntaxbaum in ein striktes und typsicheres Tupel-Format zu überführen. Die interne Struktur des AST wird in TypeScript explizit durch rekursive Typdefinitionen beschrieben:

Durch diese Definition des `AST`-Typs (siehe Listing 78) überprüft der TypeScript-Compiler, dass jeder transformierte Knoten aus einem Operator-String und einer zur jeweiligen Knotenform passenden Anzahl verschachtelter Unterknoten besteht.

```
1 export type Operator = string;
2 export type AST = string | number |
3     [Operator, AST] |
4     [Operator, AST, AST] |
5     [Operator, AST, AST, AST] |
6     [Operator, AST, AST, AST, AST];
```

Listing 78: Die typsichere Definition des AST

Syntaktische Elemente des CST, die für die spätere Programmlogik keine eigene Bedeutung besitzen, etwa Klammern, Semikolons oder Kommentare, werden durch `cst2ast` verworfen.

Bedeutung für die Gesamtarchitektur

Die Einbettung des *Lezer*-Parsers zeigt, dass dieser als Teil einer mehrstufigen Verarbeitungskette verstanden wird: Auf die formale Grammatikdefinition folgen Parsererzeugung, Analyse konkreter Eingaben in Form eines CST und schließlich die regelbasierte Transformation in eine tupelbasierte AST-Repräsentation.

Lezer übernimmt damit die Aufgabe, die textuelle Eingabe formal korrekt zu analysieren. Die fachliche Bedeutung der erkannten Konstrukte und ihre Vorbereitung für die spätere Ausführung entstehen jedoch erst durch die nachgelagerte Abbildung mittels `cst2ast`. Diese Trennung verhindert, dass spätere Auswertungsschritte mit ungeeigneten oder unerwarteten Knotenstrukturen arbeiten, und schlägt zugleich die Brücke zur *Case Study* des folgenden Kapitels, in der diese Strukturen interpretiert und ausgeführt werden.

6 Case Study: Umsetzung eines vollständigen Interpreters in TypeScript

In diesem Kapitel wird die praktische Umsetzung der zuvor entwickelten Konzepte anhand des Notebooks `03-Interpreter.ipynb` (aus „Chapter-07“ der Lehrveranstaltung) dargestellt. Im Mittelpunkt steht eine vollständige Verarbeitungskette für eine kleine C-ähnliche Sprache: von der Sprachdefinition über die Parsergenerierung und die Transformation des *Concrete Syntax Tree* (CST) in einen *Abstract Syntax Tree* (AST) bis hin zur eigentlichen Interpretation des Programms.

Die Fallstudie dient als Vergleichspunkt zwischen der ursprünglichen Python-basierten Lehrumgebung und der TypeScript-Portierung. Beide Implementierungen folgen denselben compilertechnischen Grundprinzipien, verteilen die dabei entstehende Komplexität jedoch unterschiedlich. Während *PLY* die klassische Trennung von lexikalischer und syntaktischer Analyse für den Einstieg sehr explizit macht, ermöglicht *Lezer* eine formal klarere, EBNF-nahe Darstellung der Grammatik. Dies führt bei *Lezer* jedoch zu einem detaillierten CST, der erst in einer zusätzlichen Transformationsphase in eine für die Programmauswertung geeignete Struktur überführt werden muss.

6.1 Ziel und Aufbau des Notebooks

Das Notebook implementiert einen vollständigen Interpreter für eine kleine imperative Sprache mit Variablen, arithmetischen Ausdrücken, Bedingungen, Schleifen und elementaren Funktionsaufrufen. Die Verarbeitungskette besteht in beiden Implementierungen aus vier Schritten:

1. Definition der Sprache durch eine formale Grammatik,
2. Erzeugung eines Parsers aus dieser Grammatik,
3. Konstruktion oder Ableitung eines abstrakten Syntaxbaums (AST),
4. rekursive semantische Interpretation des AST.

Unabhängig vom eingesetzten Werkzeug bleibt die zugrunde liegende Architektur eines Sprachprozessors damit erhalten. Unterschiede ergeben sich vor allem aus der Werkzeugkette, der Darstellung der Grammatik und der Frage, an welcher Stelle des Systems die strukturelle Komplexität sichtbar wird.

6.2 Sprachdefinition und Parsing im Vergleich: Python (PLY) vs. TypeScript (Lezer)

Die beiden Implementierungen unterscheiden sich weniger in den zugrunde liegenden compilertechnischen Prinzipien als in der Verteilung der einzelnen Verarbeitungsschritte. In der Python-Version mit *PLY* sind Lexer und Parser als getrennte Komponenten sichtbar. In der TypeScript-Version mit *Lezer* werden scannernahe und parsernahe Aspekte dagegen innerhalb einer gemeinsamen Grammatikbeschreibung zusammengeführt.

Zunächst zeigt Listing 79 den lexerbasierten Teil der Python-Implementierung. Die Token werden über reguläre Ausdrücke definiert und bei Bedarf bereits während des Scannens weiterverarbeitet.

```
1 import ply.lex as lex
2
3 tokens = ['NUMBER', 'IDENTIFIER', 'ASSIGN', 'IF', 'WHILE', 'EQ'] # ...
4
5 def t_NUMBER(t):
6     r'0|[1-9][0-9]*'
7     t.value = int(t.value)
8     return t
9
10 t_ASSIGN = r':='
11 Keywords = { 'if': 'IF', 'while': 'WHILE' }
12
13 def t_IDENTIFIER(t):
14     r'[a-zA-Z][a-zA-Z0-9_]*'
15     t.type = Keywords.get(t.value, 'IDENTIFIER')
16     return t
17
18 # ... weitere Definitionen für Kommentare, Leerraum und Fehlerbehandlung ...
19 lexer = lex.lex()
```

Listing 79: Auszug der PLY-Scannerdefinition (Python)

Das Listing beginnt mit dem Import von `ply.lex`. Diese Bibliothek stellt den Lexer-Generator bereit, aus dem am Ende mit `lex.lex()` ein konkreter Scanner erzeugt wird. Die Variable `tokens` legt anschließend fest, welche Tokentypen der Lexer überhaupt erzeugen darf, also etwa Zahlen, Bezeichner, Zuweisungsoperatoren und Schlüsselwörter.

Die Funktion `t_NUMBER` definiert die Regel für Zahlentoken. Der reguläre Ausdruck in der ersten Zeile beschreibt, welche Zeichenfolgen als Zahlen erkannt werden. Entscheidend ist anschließend die Zeile `t.value = int(t.value)`: Hier wird

der erkannte Text nicht nur als Token klassifiziert, sondern sofort in einen numerischen Python-Wert umgewandelt. Der Lexer übernimmt damit bereits einen ersten Verarbeitungsschritt, der über reine Tokenerkennung hinausgeht.

Mit `t_ASSIGN = r':='` wird anschließend ein fester Operator direkt als Tokenregel definiert. Die Variable `Keywords` dient dazu, bestimmte Bezeichnertexte wie `if` oder `while` nachträglich als reservierte Wörter zu markieren. Dadurch muss für Schlüsselwörter nicht zwingend eine vollständig getrennte Lexer-Regel formuliert werden.

Die Funktion `t_IDENTIFIER` erkennt zunächst alle gültigen Bezeichner mit einem regulären Ausdruck. Erst danach wird geprüft, ob der konkrete Text des Bezeichners in der Tabelle `Keywords` enthalten ist. Falls dies der Fall ist, wird der Tokentyp nachträglich angepasst. Das bedeutet: Auch in der PLY-Version besteht der Scanner nicht nur aus passiver Mustererkennung, sondern enthält bereits Logik zur Unterscheidung verschiedener sprachlicher Rollen.

Dieses Listing macht die klassische Lexer-Phase in *PLY* damit gut sichtbar. Zahlen, Bezeichner und Operatoren werden zunächst als Token erkannt. Zugleich bleibt die Verarbeitung nicht vollständig auf eine rein formale Zerlegung des Eingabetextes beschränkt, da bereits Tokenwerte umgewandelt und Bezeichner anhand ihres Inhalts spezialisiert werden.

Listing 80 zeigt anschließend den parserbasierten Teil der Python-Implementierung. Hier werden die Produktionsregeln nicht nur zur Syntaxanalyse verwendet, sondern zugleich zur direkten Konstruktion der internen Zielstruktur.

```
1 start = 'program'
2
3 def p_program_more(p):
4     "program : stmtnt program"
5     p[0] = ('.', p[1]) + p[2][1:]
6
7 def p_stmtnt_if(p):
8     "stmtnt : IF '(' bool_expr ')' stmtnt"
9     p[0] = ('if', p[3], p[5])
10
11 def p_expr_plus(p):
12     "expr : expr '+' product"
13     p[0] = ('+', p[1], p[3])
14
15 # ... weitere Regeln für Schleifen, Zuweisungen und Operatoren ...
16
17 parser = yacc.yacc(write_tables=False, debug=True)
```

Listing 80: Direkte AST-Konstruktion in PLY (Auszug)

Die Variable `start` legt fest, mit welchem Nichtterminalsymbol der Parser beginnen soll. In diesem Fall ist dies `program`. Damit weiß *PLY*, welche Regel den Einstiegspunkt für die vollständige Analyse einer Eingabedatei bildet.

Jede Funktion mit dem Präfix `p_` beschreibt anschließend eine Produktionsregel. Die Grammatik selbst steht jeweils als Docstring in der ersten Zeile der Funktion, etwa `"program : stmt program"`. Der Parameter `p` enthält dabei die Bestandteile der rechten Seite der Produktionsregel. Über `p[0]` wird festgelegt, welches Ergebnis die gesamte Produktion liefert.

Wichtig ist, dass diese Funktionen nicht nur feststellen, ob eine Eingabe syntaktisch passt. Sie bauen zugleich direkt den AST auf. In `p_expr_plus` wird aus einem Ausdruck der Form `expr + product` unmittelbar das Tupel `('+', p[1], p[3])` erzeugt. Die syntaktische Struktur und ihre interne Repräsentation entstehen hier also im selben Schritt.

Dasselbe Prinzip zeigt sich in `p_stmt_if`. Die Regel beschreibt zunächst formal eine `if`-Anweisung. Gleichzeitig wird mit `('if', p[3], p[5])` direkt der passende AST-Knoten erzeugt, der später vom Interpreter weiterverarbeitet werden kann. Die Produktionsregeln in *PLY* übernehmen damit zwei Aufgaben zugleich: Syntaxanalyse und AST-Konstruktion.

Die Funktion `p_program_more` ist zusätzlich interessant, weil sie zeigt, wie rekursive Listenstrukturen aufgebaut werden. Aus einer Anweisung `stmt` und dem restlichen Programm `program` entsteht eine Punktlistenstruktur mit dem Operator `'.'`. Diese interne Darstellung wird später für die Ausführung von Anweisungsfolgen wieder aufgegriffen.

Die letzte Zeile `parser = yacc.yacc(...)` erzeugt schließlich den Parser aus allen zuvor definierten Regeln. Auch hier zeigt sich die enge Verzahnung von Grammatik und Programmlogik: Die Regeln beschreiben nicht nur die Sprache, sondern enthalten bereits die Konstruktion der Zielstruktur.

An diesem Listing wird deutlich, dass die *PLY*-Produktionsregeln zwei Aufgaben zugleich übernehmen. Sie beschreiben einerseits, welche syntaktischen Strukturen zulässig sind, und legen andererseits unmittelbar fest, wie daraus AST-Knoten erzeugt werden. Syntaxanalyse und AST-Konstruktion sind in der Python-Version daher eng miteinander verflochten.

Die TypeScript-Implementierung mit *Lezer* organisiert diese Schritte anders. Scanner- und parsernahe Aspekte werden nicht in getrennten Artefakten implementiert, sondern innerhalb einer gemeinsamen deklarativen Grammatikbeschreibung zusammengeführt. Listing 81 zeigt den Beginn dieser Grammatik mit Startsymbol, Token-Definitionen und Skip-Regeln.

Das Listing beginnt mit dem Import von `buildParser`. Diese Funktion erzeugt aus der deklarativen Grammatikbeschreibung den später verwendeten *Lezer*-

```

1 import { buildParser } from "@lezer/generator";
2
3 const grammarString = `
4   @top Script { Program }
5
6   @tokens {
7     Identifier { ${[a-zA-Z] ${[a-zA-Z0-9_]}* }
8     Number      { "0" | ${[1-9] ${[0-9]}* }
9
10    "+" "-" "*" "/" "%"
11    "!=" "==" "!=" "<=" ">=" "<" ">"
12    "(" ")" "{" "}"
13    ";" " ",
14
15    space { ${[ \t\n\r]+ }
16    LineComment { "//" ![\n]* }
17    BlockComment { "/*" ( ![*] | "*" + ![*/] )* "*" + "/" }
18
19    @precedence { LineComment, BlockComment, "/" }
20  }
21
22  @skip { space | LineComment | BlockComment }
23
24  IF      { @specialize<Identifier, "if"> }
25  WHILE  { @specialize<Identifier, "while"> }

```

Listing 81: Beginn der Lezer-Grammatik (Startsymbol und Scanner-Token)

Parser. Anders als bei *PLY* werden Token-Definitionen und Grammatikregeln dabei nicht als verstreute Host-Code-Funktionen formuliert, sondern als zusammenhängende Spezifikation in einem einzigen String.

Die erste Zeile innerhalb des Strings, `@top Script { Program }`, legt das Startsymbol fest. Sie erfüllt damit dieselbe fachliche Rolle wie `start = 'program'` in der Python-Version, verwendet dafür aber die Lezer-spezifische Deklarations-syntax.

Der Block `@tokens` beschreibt anschließend die elementaren Tokenklassen. `Identifizier` erkennt Bezeichner, die mit einem Buchstaben beginnen und danach Buchstaben, Ziffern oder Unterstriche enthalten dürfen. `Number` beschreibt natürliche Zahlen ohne führende Nullen, außer im Sonderfall der isolierten Null. Im Unterschied zu *PLY* wird hier jedoch noch kein Text in einen konkreten Zahlenwert umgewandelt. Die Beschreibung bleibt zunächst auf der Ebene der formalen Tokenerkennung.

Die nachfolgenden Zeichenketten wie `"+"`, `:=` oder `"("` definieren feste terminale Symbole der Sprache. Leerraum und Kommentare werden ebenfalls als Tokenarten beschrieben, im Block `@skip` aber anschließend als zu ignorierende Eingabeelemente markiert. Dadurch muss die eigentliche Grammatik später nicht bei jeder Regel erneut erwähnen, dass zwischen Tokens auch Leerraum oder Kommentare stehen dürfen.

Besonders wichtig ist die Regel `@precedence { LineComment, BlockComment, "/" }`. Sie löst den Konflikt, dass das Zeichen `/` sowohl als Divisionsoperator als auch als Beginn eines Kommentars auftreten kann. Lezer soll in solchen Fällen zuerst prüfen, ob eine längere Kommentarform vorliegt, und nur andernfalls das einfache Slash-Token erzeugen.

Die Regeln `IF` und `WHILE` zeigen schließlich die Spezialisierung von Schlüsselwörtern. Zunächst wird ein passender Text als `Identifizier` erkannt. Erst danach wird geprüft, ob sein vollständiger Inhalt genau `if` oder `while` lautet. Dadurch bleiben längere Bezeichner wie `ifCounter` gültig und werden nicht fälschlich in ein Schlüsselwortpräfix und einen Rest zerlegt.

Listing 82 und Listing 83 ergänzen diese gemeinsame Spezifikation um die eigentlichen Programm-, Anweisungs- und Ausdrucksregeln und schließen mit der Parsergenerierung ab. Der daraus erzeugte Lezer-Parser verarbeitet den Eingabetext zunächst zu einem konkreten Syntaxbaum. Anders als bei *PLY* entsteht die interne Zielstruktur also noch nicht direkt in den Grammatikregeln.

Der erste Teil der Grammatik beschreibt die grundlegende Struktur vollständiger Programme und einzelner Anweisungen. Die Regel `Program` ist rechtsrekursiv formuliert: Ein Programm besteht entweder aus einer Anweisung `Stmnt`, gefolgt von einem weiteren `Program`, oder aus der leeren Zeichenkette `.` Diese Form


```

26 Program {
27     Stmt Program |
28     ""
29 }
30
31 Stmt {
32     IF "(" BoolExpr ")" Stmt |
33     WHILE "(" BoolExpr ")" Stmt |
34     "{" Program "}"
35     Identifier ":" Expr ";" |
36     Expr ";"
37 }
38
39 BoolExpr {
40     Expr "==" Expr |
41     Expr "!=" Expr |
42     Expr "<=" Expr |
43     Expr ">=" Expr |
44     Expr "<" Expr |
45     Expr ">" Expr
46 }

```

Listing 82: Fortsetzung der Lezer-Grammatik (Teil 1: Anweisungen)

entspricht einer rekursiv aufgebauten Liste von Anweisungen und bereitet bereits die spätere interne Darstellung als Punktliste vor.

Die Regel `Stmt` legt fest, welche Arten von Anweisungen in der Sprache zulässig sind. Erfasst werden bedingte Anweisungen mit `IF`, Schleifen mit `WHILE`, Blockanweisungen in geschweiften Klammern, Zuweisungen sowie Ausdrucksanweisungen. Für Studierende ist hier besonders hilfreich, dass jede Alternative unmittelbar eine konkrete Sprachkonstruktion repräsentiert. Die Grammatik kann daher nahezu wie eine formale Aufzählung der erlaubten Anweisungsformen gelesen werden.

Die Regel `BoolExpr` beschreibt boolesche Ausdrücke ausschließlich als Vergleiche zwischen zwei arithmetischen Ausdrücken. Zulässig sind also nur Formen wie `Expr == Expr` oder `Expr < Expr`. Diese Einschränkung ist didaktisch nützlich, weil sie die boolesche Auswertung klar von der arithmetischen Auswertung trennt. Anders als in vielen Vollsprachen gibt es hier keine implizite Wahrheitsinterpretation beliebiger Zahlenwerte.

Der zweite Teil der Grammatik beschreibt die Struktur arithmetischer Ausdrücke. Die Regel `Expr` erfasst Addition und Subtraktion, während `Product` für Multiplikation, Division und Modulo zuständig ist. Dass diese beiden Ebenen getrennt formuliert werden, ist kein bloßer Stil, sondern modelliert direkt die gewünschte

```

47 Expr {
48     Expr "+" Product |
49     Expr "-" Product |
50     Product
51 }
52
53 Product {
54     Product "*" Factor |
55     Product "/" Factor |
56     Product "%" Factor |
57     Factor
58 }
59
60 Factor {
61     "(" Expr ")" |
62     Number |
63     Identifier |
64     Identifier "(" ExprList ")"
65 }
66
67 ExprList {
68     NeExprList |
69     ""
70 }
71
72 NeExprList {
73     Expr |
74     Expr "," NeExprList
75 }
76 ~;
77
78 const parser = buildParser(grammarString);

```

Listing 83: Ende der Lezer-Grammatik (Teil 2: Ausdrücke)

Operatorpriorität: Multiplikation und Division binden dadurch stärker als Addition und Subtraktion.

Die Regel `Factor` beschreibt die elementaren Bausteine, aus denen größere Ausdrücke zusammengesetzt werden. Dazu gehören geklammerte Ausdrücke, Zahlen, Bezeichner und Funktionsaufrufe. Besonders wichtig ist hier der Fall Identifier `"(ExprList ")"`, da Funktionsaufrufe syntaktisch wie ein Bezeichner mit nachfolgender Argumentliste behandelt werden. Auch diese Struktur wird später nicht sofort als fertiger AST-Knoten geliefert, sondern liegt nach dem Parsing zunächst in der detaillierteren CST-Form vor.

Mit `ExprList` und `NeExprList` werden schließlich Argumentlisten beschrieben. `ExprList` erlaubt entweder eine nichtleere Liste oder die leere Zeichenkette und deckt damit auch Funktionsaufrufe ohne Argumente ab. `NeExprList` ist wiederum rekursiv formuliert: Eine nichtleere Liste besteht aus genau einem Ausdruck oder aus einem Ausdruck, gefolgt von einem Komma und einer weiteren nichtleeren Liste. Auch hier zeigt sich dasselbe Grundprinzip wie bereits bei `Program`: Wiederholungsstrukturen werden nicht durch einen separaten Listenoperator, sondern durch explizite Rekursion modelliert.

Die letzte Zeile erzeugt mit `buildParser(grammarString)` den Lezer-Parser aus der vollständigen Grammatikbeschreibung. Dieser Parser verarbeitet den Eingabetext anschließend zu einem konkreten Syntaxbaum. Anders als in der Python-Version mit *PLY* wird die interne Zielstruktur dabei jedoch noch nicht unmittelbar in den Grammatikregeln aufgebaut. Die Überführung in den später vom Interpreter verwendeten AST erfolgt erst im nachgelagerten Transformationsschritt.

6.3 Transformation vom CST zum AST

Während die Python-Version den AST bereits innerhalb der PLY-Produktionsregeln konstruiert, endet die TypeScript-Version mit *Lezer* zunächst bei einem konkreten Syntaxbaum (CST). Für die spätere Programmauswertung muss dieser Baum daher in einem zusätzlichen Schritt in einen abstrakten Syntaxbaum überführt werden.

Listing 84 zeigt diese nachgelagerte Transformationsphase. Die eigentliche syntaktische Analyse ist zu diesem Zeitpunkt bereits abgeschlossen; die Funktion `parse` verbindet nun den von *Lezer* erzeugten CST mit der projektspezifischen AST-Konstruktion.

Die Funktion `parse` bündelt drei aufeinanderfolgende Verarbeitungsschritte. Zunächst wird mit `readFileSync` der Quelltext vollständig aus der Datei eingelesen. Anschließend erzeugt `parser.parse(source)` aus diesem Text einen konkreten Syntaxbaum, dessen Struktur die vollständige Ableitung nach der Lezer-

```

1 import { readFileSync } from "fs";
2 import { cst2ast } from "./CST2AST";
3
4 const myOperators = [
5     "IF", "WHILE", ":",
6     "+", "-", "*", "/", "%",
7     "==", "!=", "<", ">", "<=", ">="
8 ];
9
10 const myListVars = ["Program", "NeExprList"];
11
12 function parse(fileName: string): AST {
13     const source = readFileSync(fileName, "utf8");
14     const tree = parser.parse(source);
15     return cst2ast(tree.cursor(), source, myOperators, myListVars);
16 }

```

Listing 84: CST-zu-AST-Transformation in TypeScript

Grammatik enthält.

Für die spätere Auswertung genügt dieser CST jedoch nicht. Er enthält noch viele syntaktische Details, die für die Interpreterlogik nicht unmittelbar benötigt werden. Deshalb ruft `parse` im letzten Schritt die Funktion `cst2ast` auf, welche den CST traversiert und daraus die kompaktere interne AST-Repräsentation erzeugt.

Die beiden Hilfslisten `myOperators` und `myListVars` steuern diese Transformation. `myOperators` legt fest, welche Symbole im AST als eigentliche Operator-knoten erscheinen sollen. `myListVars` markiert jene Grammatikvariablen, die als rekursive Listen behandelt und daher in Punktlisten überführt werden müssen.

Die TypeScript-Version führt damit einen expliziten Zwischenschritt ein, der in der Python-Version nicht als eigenständige Phase sichtbar wird. Während *PLY* Syntaxanalyse und AST-Konstruktion direkt im Parser miteinander verbindet, trennt die Lezer-basierte Implementierung die formale Analyse des Eingabetextes von der internen Modellierung des Programms. Diese zusätzliche Transformationsphase erhöht die Modularität der Implementierung, macht die Verarbeitungskette aber zugleich mehrstufiger.

6.4 Darstellung rekursiver Listenstrukturen

Bei der Modellierung rekursiver Listenstrukturen folgen beide Implementierungen demselben Grundprinzip. Sowohl Programmlisten als auch Argumentlisten werden als Punktlisten dargestellt, die an klassische *Cons*-Zellen der funktionalen Programmierung erinnern. In Python erfolgt diese Konstruktion bereits direkt in

den *PLY*-Regeln, etwa durch `p[0] = ('.', p[1]) + p[2][1:]` in Listing 80.

In der TypeScript-Version wird dieselbe Struktur nicht in der Grammatik selbst erzeugt, sondern über das in Definition 5.8 erläuterte Verfahren der Listen-Faltung aus dem CST abgeleitet. Listing 85 zeigt die Ausführung solcher Listen über eine eigene rekursive Hilfsfunktion.

```
1 let executeList = (list: AST, env: Environment) => {
2     if (list === "") return;
3
4     if (Array.isArray(list) && list[0] === '.') {
5         const head = list[1];
6         const tail = list[2];
7
8         if (head !== undefined && tail !== undefined) {
9             execute(head, env);
10            executeList(tail, env);
11        }
12    }
13 };
```

Listing 85: Verarbeitung rekursiver Listen in TypeScript

Die Funktion `executeList` verarbeitet rekursive Punktlisten. Der erste Sonderfall ist die leere Zeichenkette `''`, die das Ende einer Liste markiert. In diesem Fall gibt es keine weiteren Anweisungen auszuführen.

Im eigentlichen Rekursionsfall prüft die Funktion, ob der aktuelle Knoten ein Array mit dem Tag `"."` ist. Dieses Tag kennzeichnet eine nichtleere Liste, die aus einem Kopf und einem Rest besteht. Der Kopf enthält die aktuelle Anweisung, der Rest verweist rekursiv auf die verbleibende Liste.

Die beiden Variablen `head` und `tail` machen diese Struktur explizit sichtbar. Zuerst wird `head` durch `execute(head, env)` ausgeführt, anschließend wird die Funktion rekursiv auf `tail` angewendet. Damit wird die syntaktische Rechtsrekursion der Grammatik in eine schrittweise Abarbeitung der Anweisungsliste übersetzt.

6.5 Typisierung und interne AST-Repräsentation

Ein markanter Unterschied zwischen beiden Implementierungen liegt in der vertraglichen Absicherung des AST. Zunächst zeigt Listing 86 die Python-Variante, in der die Baumstruktur implizit bleibt.

Welche Form ein konkreter Knoten besitzt, ergibt sich in Python erst zur Laufzeit aus dem Operator-Tag an der ersten Tupel-Position. Listing 87 zeigt anschließend die TypeScript-Portierung, in der dieselbe Struktur explizit durch einen rekursiven

```

1 from typing import TypeVar
2 NestedTuple = TypeVar('NestedTuple')
3 NestedTuple = int | str | tuple['NestedTuple', ...]
4 Number      = int | float

```

Listing 86: AST-Typmodellierung in Python

Typ limitiert wird.

```

1 type Operator = string;
2 type AST = string | number |
3           [Operator, AST]           |
4           [Operator, AST, AST]      |
5           [Operator, AST, AST, AST]  |
6           [Operator, AST, AST, AST, AST];

```

Listing 87: AST-Typdefinition in TypeScript

Diese Typdefinition dokumentiert die Struktur des AST unmittelbar im Programmtext. Gleichzeitig zwingt sie die Transformationslogik dazu, nur solche Knoten zu erzeugen, die dem vereinbarten Schema entsprechen. Im Vergleich zur Python-Version verschiebt sich dadurch ein Teil der Fehlererkennung aus der Laufzeit in die statische Typprüfung.

6.6 Interpreterarchitektur und Ausführung

Die Grundstruktur des Interpreters bleibt in beiden Notebooks gleich. In beiden Fällen erfolgt eine funktionale Trennung in Anweisungsausführung (`execute`), arithmetische Auswertung (`evaluate`) und boolesche Auswertung (`evaluate_bool`).

Zunächst zeigt Listing 88 die Auswertungslogik in Python. Hier wird der Kontrollfluss elegant durch *Pattern Matching* formuliert, welches das Tupel entpackt und Variablen im selben Schritt zuweist.

Listing 89 zeigt die TypeScript-Portierung derselben Funktion. Sie folgt derselben semantischen Idee, verwendet als Dispatcher jedoch explizite Fallunterscheidungen auf Basis von Array-Prüfungen.

Im TypeScript-Code fungiert die Funktion `execute` als zentraler Dispatcher für Anweisungen. Zunächst wird geprüft, ob der übergebene Knoten überhaupt ein Array ist. Dies ist notwendig, weil der AST auch atomare Werte wie Zahlen oder Bezeichner enthalten kann, die keine ausführbaren Anweisungen darstellen.

In der Variablen `tag` wird anschließend das erste Element des Tupels gespeichert. Dieses Tag bestimmt, welche Art von Anweisung vorliegt. Der Fall `"."`

```

1 def execute(stmtnt: NestedTuple, Values: dict[str, Number]) -> None:
2     match stmtnt:
3         case ('.', *SL):
4             execute_tuple(tuple(SL), Values)
5         case (':=', var, value):
6             Values[var] = evaluate(value, Values)
7         case ('expr', expr):
8             evaluate(expr, Values)
9         case ('if', test, stmtnt):
10            if evaluate_bool(test, Values):
11                execute(stmtnt, Values)
12        case ('while', test, stmtnt):
13            while evaluate_bool(test, Values):
14                execute(stmtnt, Values)
15        case _:
16            assert False, f'{stmtnt} unexpected'

```

Listing 88: Ausführung von Anweisungen in Python

delegiert unmittelbar an `executeList`, weil Punktlisten keine einzelne Anweisung, sondern eine Folge von Anweisungen repräsentieren.

Der Fall "block" behandelt Blockanweisungen. Hier wird der im Block enthaltene Programmkörper ausgelesen und als Liste ausgeführt. Der Fall `:=` implementiert Zuweisungen, indem zunächst der Variablenname und der rechte Ausdruck extrahiert und anschließend das berechnete Ergebnis in der Umgebung gespeichert werden.

Die Fälle `if` und `while` realisieren die Kontrollflusskonstrukte der Sprache. In beiden Fällen wird zunächst die Bedingung ausgewertet. Beim `if` wird der Rumpf genau dann ausgeführt, wenn die Bedingung wahr ist; bei `while` geschieht dies wiederholt, solange die Bedingung wahr bleibt.

Der letzte Fall `call` zeigt, dass Funktionsaufrufe syntaktisch als Ausdrücke behandelt werden, aber dennoch Seiteneffekte besitzen können. Deshalb delegiert `execute` hier an `evaluate`. Die Ausführungsfunktion enthält somit selbst keine Fachlogik für `print` oder `read`, sondern überlässt diese der Ausdrucksauswertung.

Im Vergleich zur Python-Version wird damit deutlich, dass TypeScript dieselbe Logik expliziter entfalten muss. Die TypeScript-Version wirkt dadurch ausführlicher und technischer, macht ihre Kontrollflüsse und Sicherheitsprüfungen aber auch unmittelbar sichtbar.

```

1 let execute = (node: AST, env: Environment): void => {
2     if (!Array.isArray(node)) return;
3     const tag = node[0];
4
5     if (tag === '.') {
6         executeList(node, env);
7         return;
8     }
9     if (tag === "block") {
10        const content = node[1];
11        if (content !== undefined) {
12            executeList(content, env);
13        }
14    }
15    else if (tag === ":=") {
16        const name = node[1];
17        const expr = node[2];
18        if (typeof name === "string" && expr !== undefined) {
19            env.set(name, evaluate(expr, env));
20        }
21    }
22    else if (tag === "IF") {
23        const cond = node[1];
24        const body = node[2];
25        if (cond !== undefined && body !== undefined) {
26            if (evaluateBool(cond, env)) execute(body, env);
27        }
28    }
29    else if (tag === "WHILE") {
30        const cond = node[1];
31        const body = node[2];
32        if (cond !== undefined && body !== undefined) {
33            while (evaluateBool(cond, env)) execute(body, env);
34        }
35    }
36    else if (tag === "Call") {
37        evaluate(node, env);
38    }
39 };

```

Listing 89: Ausführung von Anweisungen in TypeScript

6.7 Auswertung arithmetischer Ausdrücke

Auch bei der Auswertung arithmetischer Ausdrücke bleiben die semantischen Grundideen erhalten. Literale werden direkt zurückgegeben, Variablen in der Umgebung nachgeschlagen und Operatoren rekursiv ausgewertet. Listing 90 zeigt die Python-Implementierung dieser Logik.

```
1 def evaluate(expr: NestedTuple, Values: dict[str, Number]) -> Number:
2     match expr:
3         case int():
4             return expr
5         case str():
6             return Values[expr]
7         case ('call', 'read'):
8             return int(input('Please enter a natural number: '))
9         case ('call', 'print', expr):
10            print(evaluate(expr, Values))
11            return 0
12        case ('+', lhs, rhs):
13            return evaluate(lhs, Values) + evaluate(rhs, Values)
14        # ... weitere Operatoren (-, *, /, %) ...
15        case _:
16            assert False, f'{expr} unexpected'
```

Listing 90: Arithmetische Auswertung in Python

Listing 91 zeigt anschließend die TypeScript-Portierung derselben Funktion (Teil 1). Sie bildet die rekursive Struktur exakt nach, ergänzt diese jedoch um sicherheitskritische Überprüfungen.

Im TypeScript-Code berechnet `evaluate` den numerischen Wert eines Ausdrucks. Die ersten beiden Fallunterscheidungen behandeln die einfachsten Formen von AST-Knoten: Zahlen werden direkt zurückgegeben, während Zeichenketten als Variablennamen interpretiert und in der Umgebung nachgeschlagen werden.

Ist ein Variablenname nicht in der Umgebung vorhanden, wird ein Fehler ausgelöst. Diese Prüfung ist wichtig, weil das Programm sonst mit einem impliziten oder undefinierten Wert weiterlaufen würde. Die TypeScript-Version macht diesen Fehlerfall daher ausdrücklich sichtbar.

Der nächste Schritt behandelt zusammengesetzte Ausdrucks-knoten. Liegt ein Array vor, wird zunächst wieder dessen Tag ausgelesen. Gehört dieses Tag zu den arithmetischen Operatoren, werden linker und rechter Operand bestimmt und anschließend rekursiv ausgewertet.

Erst nach dieser rekursiven Auswertung erfolgt die eigentliche Berechnung im `switch`-Block. Die Struktur der Funktion spiegelt damit direkt den rekursiven

```

1  let evaluate = (node: AST, env: Environment): number => {
2      if (typeof node === "number") return node;
3      if (typeof node === "string") {
4          const val = env.get(node);
5          if (val === undefined) throw new Error(`Undefined var ${node}`);
6          return val;
7      }
8
9      if (Array.isArray(node)) {
10         const tag = node[0];
11
12         if (["+","-","*","/","%"].includes(tag)) {
13             const leftNode = node[1];
14             const rightNode = node[2];
15
16             if (leftNode !== undefined && rightNode !== undefined) {
17                 const l = evaluate(leftNode, env);
18                 const r = evaluate(rightNode, env);
19                 switch(tag) {
20                     case "+": return l + r;
21                     case "-": return l - r;
22                     case "*": return l * r;
23                     case "/":
24                         if (r === 0) throw new Error("Division by zero");
25                         return Math.floor(l / r);
26                     case "%":
27                         if (r === 0) throw new Error("Modulo by zero");
28                         return l % r;
29                 }
30             }
31         }
32     }
33 }

```

Listing 91: Arithmetische Auswertung in TypeScript (Teil 1: Basisoperatoren)

Aufbau von Ausdrucksbäumen wider: Zuerst werden Teilbäume ausgewertet, danach wird ihr Ergebnis durch den übergeordneten Operator kombiniert. Bei Division und Modulo wird zusätzlich geprüft, ob der rechte Operand null ist, um undefinierte Rechenoperationen frühzeitig abzufangen.

Im zweiten Teil der Funktion (Listing 92) wird die Auswertung von Funktionsaufrufen (Call) behandelt.

```
32     if (tag === "Call") {
33         const fn = node[1];
34         const argsNode = node[2];
35
36         if (fn === "print" && argsNode !== undefined) {
37             if (Array.isArray(argsNode) && argsNode[0] === '.') {
38                 const actualArg = argsNode[1];
39                 if (actualArg !== undefined) {
40                     console.log(">> STDOUT:", evaluate(actualArg, env));
41                 }
42             }
43             return 0;
44         }
45         if (fn === "read") {
46             const input = inputStream.shift();
47             const val = input !== undefined ? Number(input) : 0;
48             console.log("<< STDIN:", val);
49             return val;
50         }
51     }
52 }
53 throw new Error(`Invalid expression node: ${JSON.stringify(node)}`);
54 };
```

Listing 92: Arithmetische Auswertung in TypeScript (Teil 2: Funktionsaufrufe)

Für print wird geprüft, ob überhaupt eine Argumentliste vorhanden ist und ob diese in der erwarteten Punktlistenform vorliegt. Anschließend wird das erste tatsächliche Argument extrahiert, rekursiv ausgewertet und auf der Konsole ausgegeben. Der Rückgabewert ist anschließend 0, damit print weiterhin als Ausdruck behandelt werden kann.

Der Fall read arbeitet umgekehrt. Hier wird der nächste vorbereitete Eingabewert mit inputStream.shift() aus dem simulierten Eingabestrom entnommen, in eine Zahl umgewandelt und anschließend zurückgegeben. Damit bildet die Funktion einfache Ein- und Ausgabeoperationen ab, ohne auf blockierende Konsoleingaben angewiesen zu sein.

Die abschließende Fehlermeldung am Ende der Funktion dient als Sicher-

heitsnetz. Erreicht die Auswertung einen Ausdrucks-knoten, der von keinem der vorgesehenen Fälle erfasst wird, bricht das Programm mit einer expliziten Diagnose ab. Für das Verständnis in einer Lehrumgebung ist dies hilfreicher als ein stillschweigendes Fehlverhalten.

Im Vergleich zur Python-Version wird hier sichtbar, dass die TypeScript-Portierung Laufzeitbedingungen expliziter modelliert. Fehlerfälle wie Division durch Null oder der Zugriff auf nicht initialisierte Variablen werden systematisch durch Ausnahmen behandelt, was jedoch zu einer höheren Verbosität des Codes führt.

6.8 Auswertung boolescher Bedingungen

Die boolesche Auswertung ist analog aufgebaut. Nach den Regeln der Grammatik existiert keine implizite Typkonvertierung im Sinne einer allgemeinen *Truthiness*; ein boolescher Ausdruck muss stets über einen Vergleichsoperator formuliert werden. Listing 93 zeigt die Python-Lösung.

```
1 def evaluate_bool(expr: NestedTuple, Values: dict[str, Number]) -> bool:
2     match expr:
3         case ('==', lhs, rhs):
4             return evaluate(lhs, Values) == evaluate(rhs, Values)
5         case ('!=', lhs, rhs):
6             return evaluate(lhs, Values) != evaluate(rhs, Values)
7         # ... weitere Vergleichsoperatoren (<, >, <=, >=) ...
8         case _:
9             assert False, f'{expr} unexpected'
```

Listing 93: Boolesche Auswertung in Python

Listing 94 zeigt die TypeScript-Version, die erneut das Array-Tag nutzt, um den passenden Fall zu identifizieren.

Die Funktion `evaluateBool` ist strukturell einfacher als `evaluate`, folgt jedoch demselben Grundmuster. Zunächst wird geprüft, ob überhaupt ein zusammengesetzter Knoten vorliegt und welches Tag dieser besitzt.

Nur Vergleichsoperatoren sind in dieser Funktion zulässig. Deshalb wird anschließend kontrolliert, ob das Tag zu einer der sechs erlaubten Vergleichsoperationen gehört. Ist dies der Fall, werden linker und rechter Operand rekursiv mit `evaluate` ausgewertet.

Erst auf Grundlage dieser numerischen Teilergebnisse wird im `switch`-Block der eigentliche Wahrheitswert berechnet. Die Funktion trennt damit bewusst zwischen arithmetischer Auswertung und boolescher Entscheidung. Dies macht die Interpreterlogik klarer und entspricht zugleich der Grammatik, in der boolesche Ausdrücke ausschließlich als Vergleiche definiert sind.

```

1  let evaluateBool = (node: AST, env: Environment): boolean => {
2      if (Array.isArray(node)) {
3          const tag = node[0];
4          if (["==", "!=", "<", ">", "<=", ">="].includes(tag)) {
5              const leftNode = node[1];
6              const rightNode = node[2];
7
8              if (leftNode !== undefined && rightNode !== undefined) {
9                  const l = evaluate(leftNode, env);
10                 const r = evaluate(rightNode, env);
11                 switch(tag) {
12                     case "==": return l == r;
13                     case "!=": return l != r;
14                     case "<":  return l < r;
15                     case ">":  return l > r;
16                     case "<=": return l <= r;
17                     case ">=": return l >= r;
18                 }
19             }
20         }
21     }
22     throw new Error("Invalid boolean expression");
23 };

```

Listing 94: Boolesche Auswertung in TypeScript

Im Vergleich bleibt die semantische Idee gegenüber der Python-Version unverändert. Unterschiede zeigen sich vor allem in der Art der Strukturprüfung und in der Notwendigkeit, die erwartete Form des AST vor jeder Auswertung in TypeScript explizit abzusichern.

6.9 Umgebungsmodell und Programmausführung

Der Startpunkt des Programmlaufs initialisiert die Laufzeitumgebung und den Eingabepuffer. In Python blockiert `input()` die Ausführung und wartet auf eine Konsoleneingabe (Listing 95).

```
1 def main(file):
2     with open(file, 'r') as handle:
3         program = handle.read()
4         stmtnt = yacc.parse(program)
5         Values = {}
6         execute(stmtnt, Values)
```

Listing 95: Programmausführung in Python

Listing 96 zeigt die Umsetzung in TypeScript. Hier werden asynchrone Eingabeprobleme umgangen, indem Ein- und Ausgaben vorbereitet werden.

```
1 type Environment = Map<string, number>;
2 let inputStream: string[] = [];
3
4 function runProgram(fileName: string, inputs: string[] = []) {
5     console.log(`\n--- Executing File: ${fileName} ---`);
6     console.log(`--- Inputs: [${inputs.join(", ")}] ---`);
7
8     inputStream = [...inputs];
9
10    try {
11        const ast = parse(fileName);
12        execute(ast, new Map());
13    } catch (e) {
14        console.error(e);
15    }
16 }
```

Listing 96: Programmausführung und I/O-Simulation in TypeScript

Die Typdefinition `Environment` legt fest, dass Variablennamen auf numerische Werte abgebildet werden. Mit `inputStream` wird zusätzlich ein globaler Eingabepuffer angelegt, der die spätere Simulation von `read`-Aufrufen ermöglicht.

Die Funktion `runProgram` bündelt den gesamten Ablauf eines Testlaufs. Zunächst werden Dateiname und Eingabewerte protokolliert. Anschließend wird der Eingabestrom initialisiert, indem die übergebenen Werte in das globale Array kopiert werden.

Im `try`-Block erfolgt dann die eigentliche Programmausführung. Zuerst wird der Quelltext geparkt und in einen AST transformiert. Danach startet `execute` die Interpretation mit einer frischen, zunächst leeren Variablenumgebung. Der `catch`-Block sorgt schließlich dafür, dass Laufzeit- oder Parsefehler nicht das gesamte Notebook abbrechen, sondern kontrolliert ausgegeben werden.

Im Vergleich zur Python-Version unterscheidet sich die Eingabebehandlung deutlich. Während Python blockierende Konsoleneingaben nutzt, verwendet die TypeScript-Portierung einen vorbereiteten Eingabepuffer in Form des Arrays `inputStream`. Diese Lösung ist als Workaround für die Notebook-Umgebung zu verstehen und ersetzt keine echte interaktive Standardeingabe. Gleichwohl simuliert sie einen sequentiellen Eingabestrom, wie er in praktischen Verarbeitungskontexten durchaus üblich ist. Dadurch eignet sich der Ansatz insbesondere für reproduzierbare Testläufe mit kontrollierten Eingabewerten.

6.10 Ergebnisse der Case Study

Die Fallstudie zeigt, dass beide Implementierungen dieselbe interpretertechnische Grundidee realisieren, die dabei entstehende Komplexität jedoch unterschiedlich verteilen. In der Python-Version mit *PLY* ist die klassische Trennung von Scanner und Parser leicht nachvollziehbar, da die einzelnen Phasen der Sprachverarbeitung klar voneinander abgegrenzt bleiben. Die Interpreterlogik selbst lässt sich durch Pattern Matching über dynamische Tupel äußerst kompakt ausdrücken. Gleichzeitig wirkt die konkrete Implementierung der Grammatik im Quelltext vergleichsweise unübersichtlich, weil sich formale Regeln auf zahlreiche Python-Funktionen und Docstrings verteilen.

Die TypeScript-Version mit *Lezer* verschiebt diesen Schwerpunkt. Die deklarative Grammatik ist deutlich näher an der formalen Notation aus der theoretischen Informatik und daher in ihrer Oberflächenstruktur geschlossener und leichter lesbar. Dem steht jedoch gegenüber, dass der generierte CST für die eigentliche Programmauswertung zunächst noch zu detailliert ist. Der zentrale Mehraufwand der Portierung liegt daher in der zusätzlichen Transformation des durch *Lezer* erzeugten CST in einen auswertbaren AST. Erst durch die in `cst2ast` gekapselte Schicht aus Terminal-Extraktion, Infix Operator Promotion und Listen-Faltung entsteht eine interne Repräsentation, die sich im typisierten Interpreter weiterverarbeiten lässt.

Konsequenzen für die Lehrveranstaltung

Gerade diese Verschiebung der Komplexität erweist sich für den Einsatz in der Lehrveranstaltung als vorteilhaft. Die aufwendige `cst2ast`-Transformation kann als gekapselte Komponente über eine feste Schnittstelle bereitgestellt werden, sodass die Studierenden sich nicht unmittelbar mit allen implementierungsspezifischen Details der Baumerstellung befassen müssen.

Der Fokus kann dadurch stärker auf die deklarative, EBNF-nahe Grammatikdefinition gelegt werden. Im Unterschied zur Python-Variante, in der formale Grammatikregeln und operative AST-Konstruktion enger miteinander verflochten sind, erlaubt die TypeScript-Version eine klarere Trennung zwischen Sprachbeschreibung und späterer Programmauswertung. Die Portierung ist damit nicht nur eine syntaktische Übertragung des bestehenden Beispiels, sondern eine strukturelle Neuordnung, die den Zugang zu den zentralen Konzepten der Vorlesung erleichtert.

7 Fazit

Ziel dieser Arbeit war die Übertragung der interaktiven Python-basierten Lehrnotebooks der Vorlesung „Theoretische Informatik III“ in das TypeScript-Ökosystem. Dabei wurde bewusst nicht nur eine syntaktische Übersetzung angestrebt, sondern eine strukturelle Neuausrichtung der zugrunde liegenden Architektur. Im Zentrum stand die Frage, inwiefern sich Konzepte formaler Sprachen durch statische Typisierung präziser modellieren und zuverlässiger umsetzen lassen.

Die Ergebnisse der Arbeit zeigen deutlich, dass TypeScript durch sein statisches Typsystem einen wesentlichen Mehrwert für die Modellierung komplexer Datenstrukturen bietet. Während Python viele Fehler erst zur Laufzeit sichtbar macht, ermöglicht TypeScript eine frühzeitige Validierung von Invarianten bereits während der Entwicklung. Insbesondere bei verschachtelten Strukturen wie Syntaxbäumen oder Zustandsmengen trägt dies maßgeblich zur Robustheit der Implementierung bei. Damit wird ein zentraler Vorteil statischer Typisierung bestätigt, nämlich die Vorverlagerung von Fehlererkennung in die Entwicklungsphase [8, S. 371ff].

Ein zentrales Ergebnis der Arbeit ist die Erkenntnis, dass eine naive Portierung nicht ausreicht. Unterschiede in der Laufzeitsemantik, wie etwa das referenzbasierte Verhalten von Mengen in JavaScript, erfordern eigenständige architektonische Lösungen. Die entwickelte Datenstruktur `RecursiveSet` zeigt exemplarisch, wie durch gezielte Implementierungsentscheidungen mathematische Konzepte wie Wertgleichheit und Immutabilität in TypeScript nachgebildet werden können. Gleichzeitig verdeutlicht sie, dass Typsysteme allein nicht alle semantischen Unterschiede zwischen Programmiersprachen überbrücken können, sondern durch geeignete Datenstrukturen ergänzt werden müssen.

Auch im Bereich der Sprachverarbeitung konnte gezeigt werden, dass die grundlegenden Prinzipien der Compilertechnik unabhängig von der Zielplattform bestehen bleiben. Die Transformation von der lexikalischen Analyse über die syntaktische Analyse bis hin zur semantischen Interpretation folgt sowohl in der Python- als auch in der TypeScript-Implementierung denselben theoretischen Grundlagen [8, S. 4ff]. Der Einsatz von Lezer als Parsergenerator verdeutlicht jedoch, wie sich diese klassischen Konzepte an moderne Anforderungen wie inkrementelles Parsing und editornahe Anwendungen anpassen lassen.

Die Case Study zur Implementierung eines vollständigen Interpreters zeigt schließlich, dass sich die gesamte Verarbeitungskette – von der Grammatikdefinition bis zur Ausführung – erfolgreich in TypeScript abbilden lässt. Dabei wird deutlich, dass die statische Typisierung nicht nur der Fehlervermeidung dient, sondern auch die Struktur und Lesbarkeit des Codes verbessert. Gleichzeitig bleibt

ein gewisser Trade-off bestehen: Eine vollständig präzise Typmodellierung kann zu erhöhter Komplexität führen und erfordert bewusste Entwurfsentscheidungen zwischen Strenge und Flexibilität.

Zusammenfassend lässt sich festhalten, dass die Übersetzung der Lehrmaterialien nach TypeScript nicht nur technisch möglich, sondern auch didaktisch sinnvoll ist. Die stärkere Typisierung unterstützt das Verständnis formaler Strukturen, zwingt zur expliziten Modellierung von Invarianten und reduziert typische Fehlerquellen dynamischer Systeme. Gleichzeitig zeigt die Arbeit, dass eine erfolgreiche Migration stets eine konzeptionelle Anpassung erfordert und nicht auf eine reine Syntaxübertragung reduziert werden kann.

Die entwickelten Ansätze bilden damit eine solide Grundlage für zukünftige Erweiterungen, etwa die weitergehende Typisierung von AST-Strukturen oder die Integration zusätzlicher Analyse- und Optimierungsschritte innerhalb der Interpretipeline.

8 Literaturverzeichnis

- [1] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019 (Revision of IEEE 754-2008). IEEE, Juli 2019. URL: https://www-users.cse.umn.edu/~vinals/tspot_files/phys4041/2020/IEEE%20Standard%20754-2019.pdf.
- [2] Georg Cantor. „Beiträge zur Begründung der transfiniten Mengenlehre“. In: *Mathematische Annalen* 46 (1895), S. 481–512. URL: <http://eudml.org/doc/157768>.
- [3] Austin Appleby. *MurmurHash3*. <https://chromium.googlesource.com/external/smhasher/+58dd8869da8c95f5c26ec70a6cdd243a7647c8fc/MurmurHash3.cpp>. C++ Referenzimplementierung. Chromium-Mirror, abgerufen am 08.04.2026. 2011.
- [4] Richard Fabian. *Data-Oriented Design: Software Engineering for Limited Resources and Short Schedules*. Frei zugängliche PDF-Version. Richard Fabian, 2018. ISBN: 978-1916478701. URL: <https://dataorienteddesign.com/dodbook.pdf>.
- [5] Thomas H. Cormen u. a. *Introduction to Algorithms*. 4. Aufl. Abgerufen am 08.04.2026. MIT Press, 2022. URL: <https://iitdh.ac.in/~konjengbam.and/courses/daa/CLRS4e2022.pdf>.
- [6] Pedro Celis. *Robin Hood Hashing*. Techn. Ber. CS-86-14. Frei zugängliches PDF. University of Waterloo, Department of Computer Science, 1986. URL: <https://cs.uwaterloo.ca/research/tr/1986/CS-86-14.pdf>.
- [7] David Beazley. *PLY (Python Lex-Yacc)*. 2026. URL: <https://www.dabeaz.com/ply/> (besucht am 08.02.2026).
- [8] Alfred V. Aho u. a. *Compilers: Principles, Techniques, and Tools*. 2. Aufl. Abgerufen am 09.04.2026. Addison-Wesley, 2006. ISBN: 978-0321486813. URL: <https://dpvipracollege.ac.in/wp-content/uploads/2023/01/Alfred-V.-Aho-Monica-S.-Lam-Ravi-Sethi-Jeffrey-D.-Ullman-Compilers-Principles-Techniques-and-Tools-Pearson-Addison-Wesley-2007.pdf>.
- [9] *Lezer Documentation*. CodeMirror / Lezer Project. 2026. URL: <https://lezer.codemirror.net/docs/> (besucht am 08.02.2026).
- [10] Marijn Haverbeke. *Lezer*. 3. Sep. 2019. URL: <https://marijnhaverbeke.nl/blog/lezer.html> (besucht am 08.02.2026).

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 1: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.